



دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر



تخمین زاویه ورود با استفاده از شبکه های عصبی

احمد رضا شهسواری

۸۱۰۱۹۹۴۴۵

استاد راهنما:
دکتر علی الفت

شهریور ماه ۱۴۰۳

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

چکیده¹

تشخیص و مکان یابی رویدادهای صوتی، مسئله شناسایی رخداد‌های صوتی مختلف و تخمین زدن موقعیت‌های زمانی و مکانی آنها می‌باشد. اخیراً برای حل چنین مسائلی، از روش‌های مختلف یادگیری عمیق استفاده می‌شود درحالی‌که تشخیص رویداد صوتی غالباً بر روی الگوهای زمان-فرکانسی برا ایجاد تمایز بین کلاس‌های مختلف صدا تکیه دارد، اما در تخمین زاویه ورود معمولاً از اختلاف‌های اندازه و یا فاز بین میکروفون‌ها برای تخمین جهت منابع استفاده می‌شود، به همین دلیل حل بهینه همزمان این دو مسئله ممکن است دشوار باشد و استفاده از یک شبکه عصبی برای تخمین هر دو مورد، کارایی را کاهش می‌دهد. به همین دلیل در این پژوهش از یک ساختار دو مرحله‌ای برای یادگیری عمیق استفاده میشود که از یک ویژگی ورودی جدید برای یادگیری استفاده می‌کند، همچنین برای معماری شبکه عصبی هم از یک ساختار متفاوت که نیاز به زمان یادگیری کمتری دارد و بسیار سریع‌تر عمل میکند، استفاده می‌شود که تا به حال در پژوهش دیگری برای تخمین زاویه ورود استفاده نشده است.

کلمات کلیدی:

تشخیص و مکان یابی رویدادهای صوتی، یادگیری عمیق، شبکه‌های عصبی،

تخمین زاویه ورود، استخراج ویژگی

¹ Abstract

فهرست مطالب

فصل ۱: مقدمه و بیان مساله	1
۱-۱- مقدمه	2
۱-۲- ساختار پایان نامه	4
فصل ۲: مفاهیم اولیه و پیش زمینه	5
۲-۱- مقدمه	6
۲-۲- ویژگی های ورودی	8
۲-۳- معماری MobileNet	15
فصل ۳: شبیه سازی	19
۳-۱- مقدمه	20
۳-۲- شبیه سازی ویژگی های ورودی	20
۳-۳- پیاده سازی شبکه عصبی	21
۳-۴- معیار ارزیابی	23
۳-۵- نتایج بدست آمده	25
فصل ۴: مراجع	33
پیوست ها	36

فهرست علائم اختصاری

SED	Sound Event Detection
SEL	Sound Event Localization
SELD	Sound Event Localization and Detection
DOA(E)	Direction of Arrival (Estimation)
TDOA	Time Difference of Arrival
RDOA	Relative Distance of Arrival
GCC-PHAT	Generalized Cross Correlation with Phase Transform
STFT	Short-time Fourier Transform
IPD	Inter-channel Phase Difference
EPV	Eigenvector-based Phase Vector
DWS	Depthwise Separable
IR	Impulse Response

فصل ۱

فصل ۱: مقدمه و بیان مساله

در این فصل نخست به بیان مقدمات کار، تاریخچه‌ای کوتاه از مساله تحقیق و روش کلی تحقیق پرداخته، سپس مساله و موضوع مورد بررسی در این پایان‌نامه بیان می‌شود و در نهایت به ساختار پایان‌نامه‌ی پیش رو اشاره خواهد شد.

۱-۱- مقدمه

تشخیص رویداد صدا (SED) یک حوزه در حال گسترش می باشد که در آن سعی می شود تا صداهای مختلف در محیط های شهری و طبیعی شناسایی و آنالیز شوند و در هر فریم برچسب های از پیش تعریف شده به منابع صدای فعال اختصاص داده شوند. در مقایسه با برچسب گذاری صداها، در اینجا علاوه بر تشخیص نوع صداها، سعی میشود تا زمان رخ دادن آنها هم تخمین زده شود. رویکردهای مختلفی از جمله روش های پردازش سیگنال و یادگیری ماشین برای حل این مسئله ارائه شده است که یکی از رویکردهایی که زیاد مورد استفاده قرار گرفته است، استفاده از یادگیری عمیق و ساختارهای مختلفی مانند FCNNs و CNNs و CRNNs و ResNets می باشد. در این روش ها، از نمایش های مناسب زمان-فرکانسی برای صدا استفاده می شود که مشابه ورودی های تصویر در بینایی کامپیوتر می باشند

موقعیت یابی رویداد صدا (SEL) بر شناسایی موقعیت منابع صدا تمرکز دارد و در آن سعی می شود تا مکان یک یا چند منبع صدا نسبت به یک نقطه مرجع دلخواه (که معمولاً مکان آرایه میکروفون های در حال ضبط می باشد)، بر مبنای سیگنال های صوتی چندکاناله ضبط شده، تخمین زده شود. در بیشتر مسائل عملی، هدف فقط تخمین زاویه ورود (DOA) می باشد که در آن فاصله تا آرایه میکروفون ها مدنظر نیست و فقط زاویه های azimuth و elevation تخمین زده می شوند. از کاربردهای این مسئله میتوان به شنوایی رباتیک، بهبود گفتار، کنترل نویز، جداسازی منابع و ... اشاره کرد. از جمله روش های سنتی پردازش سیگنال برای تخمین زاویه ورود میتوان به روش های TDOA، MUSIC، GCC-PHAT اشاره کرد. از جمله ایرادات این روش ها، کاهش کارایی در حضور نویز و در محیط های دارای انعکاس زیاد و نیز هزینه محاسباتی بالا می باشد، به همین دلیل امروزه برای حل این مسائل، سعی می شود تا از روش های برپایه یادگیری عمیق استفاده شود که نشان داده شده است که محیط های پرچالش دارای عملکرد بهتری می باشند. تا به حال برای ویژگی های ورودی شبکه های عصبی، برای اینکه هم از مزایای یادگیری عمیق و هم از مزایای روش های بر پایه پردازش سیگنال بهره برده شود [1]، از ویژگی های مختلفی از جمله طیف اندازه و فاز (ویژگی های برپایه طیف)، ویژگی های برپایه GCC (که به خوبی اطلاعاتی را درمورد تفاوت های زمانی نشان میدهند)، بردارهای ویژه ماتریس همبستگی فضایی، ویژگی های بین کانالی و ... استفاده شده است که همانطور که ذکر شد باعث بهبود عملکرد در تخمین زاویه ورود می شوند.

در کاربردهای عملی، یک رویداد صدا همواره در یک یا چند جهت انتقال می یابد، به همین علت منطقی بنظر می رسد که بخواهیم هر دو وظیفه تشخیص و موقعیت یابی را بطور همزمان انجام دهیم، در نتیجه در حال حاضر مسئله تشخیص و مکان یابی رویداد صوتی (SELD) می باشد که در آن نه تنها زاویه ورود، بلکه نوع و اطلاعات زمانی صدا هم شناسایی می شوند، بنابراین در این سیستم اهداف شناسایی حضور رویداد های صوتی، کلاس بندی آنها با کمک برجسب های متنی و تخمین زاویه ورود می باشد. دو مولفه اصلی این سیستم در پاراگراف های قبلی توضیح داده شده اند.

نتایج آزمایش های مختلف نشان میدهد که SED میتواند در کارایی تخمین زاویه ورود تاثیر بگذارد اگرچه آموزش همزمان SED و DOAE کارایی هر دو را کاهش می دهد، به همین دلیل برای حل این مشکل در این پژوهش از یک ساختار دو مرحله ای استفاده می شود [2]. در این روش به دو مسئله تشخیص و مکان یابی رویداد صدا در دو مرحله رسیدگی می شود، مرحله SED (در ساختار معرفی شده متناظر با شاخه SED) و مرحله DOAE (در ساختار معرفی شده متناظر با شاخه DOAE). در فرآیند آموزش، ابتدا شاخه SED فقط برای SED آموزش داده می شود، پس از آن لایه های ویژگی تعلیم داده شده به شاخه DOAE منتقل می شوند، بنابراین این شاخه، فرآیند یادگیری را با این مقادیر بعنوان مقادیر اولیه شروع میکند و از SED های داده مرجع بعنوان یک ماسک، برای آموزش DOAE استفاده می شود. در مرحله استنتاج و تست، ابتدا شاخه SED، پیش بینی های رویدادهای صدا را تخمین زده و از آنها بعنوان ماسک برای شاخه DOAE استفاده میشود تا در نهایت زاویه های ورود تخمین زده شوند. نتایج بدست آمده نشان میدهند که استفاده از پیشبینی های SED برای تخمین زاویه ورود، مفید واقع می شوند، بنابراین با طی کردن فرآیند آموزش و استنتاج در دو مرحله، کارایی SED و DOAE، هر دو بهبود می یابند.

برای معماری شبکه عصبی در سیستم دو مرحله ای ذکر شده در [2] از ساختارهای CNN و CRNN استفاده شده است، میتوان از معماری های دیگری همچون ResNet هم استفاده کرد، اما نکته ای که باید در نظر داشت، این است که در این ساختارها تعداد پارامترها بسیار زیاد بوده و فرآیند آموزش دادن آنها بسیار زمانبر می باشد، از این رو در این پژوهش از معماری MobileNet که در [3] برای کاربرد بینایی کامپیوتر معرفی شده است، استفاده میشود. در این ساختار تعداد پارامترها و عملیات های جمع و ضرب نسبت به ساختارهای ذکر شده قبلی کمتر می باشد؛ تغییر یافته این ساختار در [4] برای کاربرد برجسب گذاری صداها استفاده شده است، اما تا به حال در پژوهش دیگری برای SELD مورد استفاده قرار نگرفته است.

در نهایت برای ویژگی ورودی شبکه از ویژگی مشابه ویژگی هیستوگرام [2] استفاده می شود که

نسبت به ویژگی های ذکر شده قبلی دارای عملکرد بهتری می باشد؛ البته روابط در [2] برای حالتی هستند که از آرایه میکروفون صفحه ای استفاده کنیم و منابع صدا نیز در همان صفحه متناظر با این آرایه باشند، به عبارتی فقط زاویه azimuth تخمین زده می شود، اما در اینجا نیاز است که حالت کلی تری را در نظر بگیریم که روابط مربوطه در [5] بیان شده اند.

۲-۱- ساختار پایان نامه

فصل دوم، مقدمات، مفاهیم اولیه و پیش زمینه هایی جهت درک هر چه بهتر موضوع های مطرح شده ارائه می شود.

فصل سوم در برگیرنده توضیحات مربوط به ساختار سیستم و شبیه سازی آن و بررسی نتایج بدست آمده است.

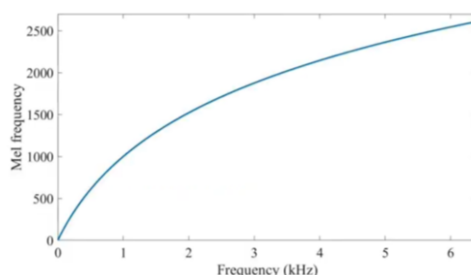
فصل ۲

فصل ۲ : مفاهیم اولیه و پیش زمینه

در فصل پیش رو مقدمات، مفاهیم اولیه و پیش زمینه‌هایی جهت درک هر چه بهتر موضوع‌های مطرح شده ارائه می شود.

۱-۲- مقدمه

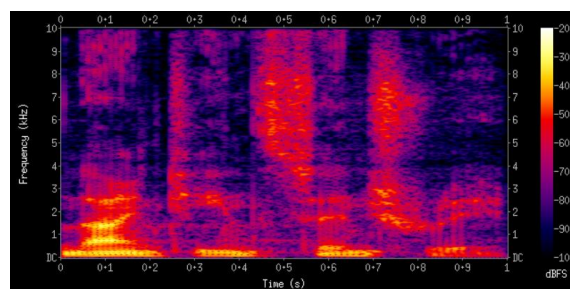
باید توجه داشت که به دلیل سائز ابعاد کوچکتر و تاکید بیشتر بر روی باندهای فرکانسی پایین تر، جایی که بیشتر اطلاعات و محتویات سیگنال در آن قرار دارند، در بیان ها و نمایش های برحسب طیف، در بیشتر مقالات بجای مقیاس فرکانسی خطی، از اسکیل فرکانسی mel استفاده می شود، در این مقیاس بازها و رزولوشن فرکانسی، در فرکانس های کوچکتر بیشتر است و هرچه فرکانس بیشتر شود، رزولوشن نیز کمتر می شود(در حالیکه در مقیاس خطی تعداد بین های فرکانسی در همه جا یکسان می باشد)، این کار به نوعی مشابه عملکرد شنوایی انسان است(که بطور لگاریتمی فرکانس را درک می کنند)، بنابراین فاصله های یکسان در مقیاس mel دارای فاصله ادراکی یکسانی می باشند



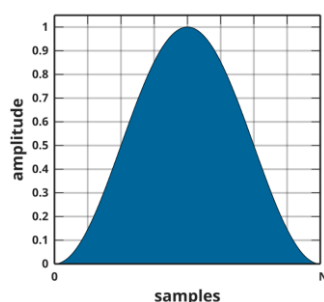
شکل (۱-۲) مقیاس mel در مقابل مقیاس خطی

مورد دیگری که باید در نظر داشت این است که برای بیان سیگنال ها در حوزه فرکانس، از تبدیل فوریه عادی DFT استفاده نمی شود، در این حالت ما اطلاعات فرکانسی برای کل سیگنال را خواهیم داشت اما از اینکه هر کدام از مولفه های فرکانسی در کدام یک از بازه های زمانی سیگنال حضور داشته اند، بی اطلاع خواهیم بود، بنابراین برای اینکه بدانیم در طول سیگنال مقادیر این مولفه های فرکانسی چگونه تغییر میکنند، نیاز است که سیگنال کلی را به قسمت های کوچکتری تقسیم کرده (که برای این کار از پنجره گذاری استفاده می شود) و بر روی هر یک از این فریم ها از تبدیل فوریه استفاده کنیم، به انجام چنین کاری تبدیل فوریه زمان-کوتاه (STFT) گفته می شود که از اعمال تابع پنجره بر روی قسمت موردنظر سیگنال و سپس محاسبه تبدیل فوریه آن، بدست می آید، بنابراین به نوعی به یک نمایش دو بعدی خواهیم رسید که اطلاعات فرکانسی در هر زمان (فریم) نمایش می دهد(حاصل STFT سیگنال $x(\tau)$ با $X(t, f)$ نمایش داده میشود). لازم به ذکر است که هرچه سائز فریم (پنجره) بزرگتر گرفته شود رزولوشن فرکانسی بیشتر اما رزولوشن زمانی کمتر می شود، همچنین برای تابع پنجره معمولاً از تابع مستطیلی بدلیل ناپیوستگی های در دو گوشه استفاده نمیشود و بجای آن غالباً از پنجره hann استفاده می شود، بنابراین سیگنال خروجی در دو

طرف به سمت صفر میل میکند و از ناپیوستگی در گوشه ها جلوگیری می شود (ناپیوستگی در گوشه ها خود را بعنوان مولفه های فرکانسی بزرگی که در سیگنال اصلی وجود ندارند، نشان میدهد که به آن نشت طیفی گفته می شود)، البته باید توجه داشت که با این نوع پنجره گذاری اگرچه ناپیوستگی های بین دو لبه (و با لبه فریم بعدی) از بین می رود اما به دلیل صفر شدن و به سمت صفر میل کردن مقادیر سیگنال در گوشه بعد از اعمال پنجره hann، عملا ما اطلاعات در این نواحی را از دست خواهیم داد، برای حل این مشکل فریم ها را دقیقا یکی پس از دیگری و بدون داشتن همپوشانی در نظر نمیگیریم، بلکه فریم بعدی طوری انتخاب میشود تا با فریم قبلی دارای همپوشانی باشد، در این حالت اگر اطلاعات را در لبه یک فریم از دست بدهیم، بدلیل اینکه این محتویات آن در فریم بعدی هم قرار دارند (چون در جایی در میانه های فریم بعدی و نه در لبه ها قرار دارند)، بنابراین اطلاعات آن ها را از دست نخواهیم داد؛ از این رو یک پارامتر جدید هم به نام hop length (یا hop size) تعریف می شود که بیانگر تعداد سمپل هایی است که به راست شیفت میخوریم تا به فریم بعدی برسیم (فاصله بین نقطه شروع دو فریم متوالی).



شکل (۲-۲) مثالی از اعمال STFT بر روی یک سیگنال^۱



شکل (۲-۳) پنجره hann

^۱ https://en.wikipedia.org/wiki/Short-time_Fourier_transform

۲-۲- ویژگی های ورودی

همانطور که در بخش های قبلی ذکر شد، یکی از ویژگی هایی که در بیشتر مقالات برای شبکه های عصبی استفاده شده است GCC-PHAT می باشد، در اینجا هم قبل از اینکه به بیان ویژگی جدید بپردازیم، سعی می شود تا روابط و ایرادات روش ذکر شده را بیان کنیم [6].

اگر سیگنال های دریافت شده توسط دو تا از سنسورها را بصورت زیر نمایش بدهیم: (سیگنال ها حقیقی و فرآیندهای تصادفی تواما ایستادن می باشند، همچنین سیگنال با دو نویز نوشته شده ناهمبسته فرض می شود، همچنین توجه شود که روابط اولیه و نمادهای استفاده شده برای این قسمت، متفاوت با روابط مربوط به ویژگی جدید در صفحات بعد، هستند)

$$x_1(t) = s_1(t) + n_1(t) \quad (۱)$$

$$x_2(t) = \alpha s_1(t + D) + n_2(t) \quad (۲)$$

که D نشاندهنده اختلاف زمانی زاویه ورود (TDOA) سیگنال در در یک سنسور نسبت به زمان ورود سنسور دیگر می باشد، هدف تخمین این تاخیر زمانی است. یک راه معمول محاسبه آرگومان تاخیر زمانی است که تابع همبستگی متقابل بین دو سیگنال یعنی $R(\tau) = E[x_1(t)x_2(t - \tau)]$ را ماکزیمم می کند، مشخص است که زمانی این حاصل بیشینه می شود که دو سیگنال یکسان (ضرایب تضعیف پشت آنها تاثیری ندارد) داشته باشیم و باتوجه به روابط نوشته شده در حالتی که نویز را صفر فرض کنیم این اتفاق در $\tau = D$ رخ خواهد داد؛ اگرچه در اغلب اوقات باتوجه به زمان مشاهده محدود، فقط میتوان از تخمینی از تابع همبستگی متقابل استفاده کرد $\hat{R}(\tau) = \frac{1}{T-\tau} \int_{\tau}^T x_1(t)x_2(t - \tau)dt$. برای افزایش دقت در تخمین میتوان سیگنال ها را فیلتر کرد و یا به عبارتی در حوزه فرکانس آنها را در یک تابع وزن ضرب کنیم که به این تکنیک GCC گفته می شود. میدانیم که تابع همبستگی متقابل از طریق تبدیل فوریه با تابع چگالی طیف توان متقابل در ارتباط است، یعنی داریم:

$$R(\tau) = \int_{-\infty}^{\infty} G(f)e^{j2\pi f\tau}df \quad (۳)$$

و همانطور که گفته شد اگر دو سیگنال را به ترتیب از دو فیلتر $h_1(t)$ و $h_2(t)$ عبور دهیم تا به خروجی های $y_1(t)$ و $y_2(t)$ برسیم، داریم:

$$G_{y_1y_2}(f) = H_1(f)H_2^*(f)G_{x_1x_2}(f) \quad (۴)$$

بنابراین اگر تابع وزن را با $\psi_g(f) = H_1(f)H_2^*(f)$ نمایش دهیم، تابع همستگی متقابل تعمیم یافته بصورت زیر بدست می آید: (در حالت تقریبی $\hat{R}_{y_1y_2}^{(g)}(\tau)$ و $\hat{G}_{x_1x_2}(f)$ به دلیل مشاهدات محدود)

$$R_{y_1y_2}^{(g)}(\tau) = \int_{-\infty}^{\infty} \psi_g(f) G_{x_1x_2}(f) e^{j2\pi f\tau} df \quad (5)$$

توجه شود که برای سیگنال های دریافت شده در روابط قبلی، در حالت ایده آل میتوان نوشت:

$$R_{x_1x_2}(\tau) = \alpha R_{s_1s_1}(\tau - D) + R_{n_1n_2}(\tau) \quad (6)$$

بنابراین با محاسبه تبدیل فوریه این رابطه خواهیم داشت:

$$G_{x_1x_2}(f) = \alpha G_{s_1s_1}(f) e^{-j2\pi fD} + G_{n_1n_2}(f) \quad (7)$$

اگر $n_1(t)$ و $n_2(t)$ را ناهمبسته در نظر بگیریم ($G_{n_1n_2}(f) = 0$) بدست می آید:

$$R_{x_1x_2}(\tau) = \alpha R_{s_1s_1}(\tau) * \delta(\tau - D) \quad (8)$$

که همانطور که گفتیم در $\tau = D$ بیشینه رخ می دهد، البته این رابطه برای حالتی است که یک تاخیر داشته باشیم، اما اگر سیگنال دچار چند تاخیر شود، خواهیم داشت:

$$R_{x_1x_2}(\tau) = \alpha R_{s_1s_1}(\tau) * \sum_i \alpha_i \delta(\tau - D_i) \quad (9)$$

بنابراین با کانالو شدن $R_{s_1s_1}(\tau)$ عملاً تشخیص قله ها بسیار سخت می شود، به همین دلیل نیاز است که از تابع وزن مناسبی استفاده شود تا قله ها را بطور راحتتری بتوان تشخیص داد، البته قله های تیزتر به خطاهای بدیل زمان مشاهده محدود و نویز در SNR های پایین حساس تر هستند به همین دلیل هم از شبکه های عصبی استفاده میکنیم تا در برابر این موارد پایدار بماند. در [6] توابع وزن مختلفی معرفی شده که یکی از آنها که دارای عملکرد نسبتاً مناسبی است، PHAT می باشد که در آن چگالی طیف را بر اندازه آن تقسیم میکنیم، به عبارتی $\psi_p(f) = \frac{1}{|G_{x_1x_2}(f)|}$ انتخاب می شود (ثابت می شود که در محیط دارای نویز انعکاسی کم، تقریبی برای تخمینگر ML خواهد بود) و خواهیم داشت:

$$\hat{R}_{y_1y_2}^{(p)}(\tau) = \int_{-\infty}^{\infty} \frac{\hat{G}_{x_1x_2}(f)}{|G_{x_1x_2}(f)|} e^{j2\pi f\tau} df \quad (10)$$

که در حالت ایده آل (تقریب برابر مقدار اصلی باشد) و با فرض نویزهای ناهمبسته باتوجه به اینکه

$$|G_{x_1x_2}(f)| = \alpha G_{s_1s_1}(f), \text{ بنابراین } \frac{\hat{G}_{x_1x_2}(f)}{|G_{x_1x_2}(f)|} \text{ برابر } e^{-j2\pi fD} \text{ می شود و خواهیم داشت:}$$

$$R_{y_1 y_2}^{(p)}(\tau) = \delta(t - D) \quad (11)$$

بنابراین یک پیک در $\tau = D$ خواهیم داشت، البته در حالت تقریبی (و یا داشتن چند تاخیر) حاصل دقیقاً برابر تابع ضربه نمی شود، همچنین با توجه به نوع انتخاب تابع وزن، در جاهایی که توان سیگنال کوچک باشد میزان خطاها افزایش می یابد. باتوجه به توضیحات ذکر شده برای استفاده از GCC-PHAT بعنوان ویژگی ورودی، برای هر فریم صوت و بین هر دو جفت میکروفن i و j ویژگی باتوجه به تعریف طیف توان متقابل (بطور مثال اگر داشته باشیم $R_{x_1 x_2}(\tau) = x_1(\tau) * x_2^*(-\tau)$ در این صورت با تبدیل فوریه گرفتن از طرفین خواهیم داشت $G_{x_1 x_2}(f) = X_1(f)X_2^*(f)$) بصورت زیر محاسبه میشود:

$$GCC_{i,j}(t, \tau) = \mathcal{F}_f^{-1} \left[\frac{X_i(t, f)X_j^*(t, f)}{|X_i(t, f)X_j^*(t, f)|} \right] \quad (12)$$

که در آن $\mathcal{F}_f^{-1}$ تبدیل فوریه معکوس (IFFT) از f به τ می باشد و $X_i(t, f)$ حاصل STFT سیگنال میکروفون i ام $(x_i(\tau))$ است و همانطور که گفته شد تاخیر زمانی (time lag) بین دو میکروفون به کمک ماکزیمم کردن این حاصل تخمین زده میشود البته این تخمین بخصوص در شرایطی که SNR کم باشد و یا محیط دارای انعکاس های زیاد و منابع دیگری باشد، خیلی دقیق نخواهد بود. برای کاربرد SELD باتوجه به اینکه نیاز به تشخیص نوع صدا و... نیز داریم، باتوجه به توضیحاتی که در قسمت های قبلی داده شد، معمولاً در کنار ویژگی بالا، طیف log-mel نیز به ورودی شبکه داده می شود که بطور مثال بصورت زیر محاسبه می شود: (k اندیس mel و W_{mel} فیلتر mel است)

$$MELSPEC(t, k) = \log(|X(t, f)|^2 \cdot W_{mel}(f, k)) \quad (13)$$

برای ورودی ساختار هم بطور معمول تمام این ویژگی ها برای همه کانال ها با میانگین صفر و واریانس یک نرمالیزه می شوند.

حال به سراغ معرفی ویژگی جدید می رویم. باید توجه داشت که ذخیره کردن دو ویژگی بالا در کنار هم اطلاعات ضمنی مربوط به DOA در ویژگی های قبلی را به خطر می اندازد چراکه نیاز است تا time-lag های طولانی تر بر روی GCC-PHAT کوتاه بشوند به این دلیل که بعد time-lag مربوط به این ویژگی GCC-PHAT دارای نگاشت محلی یک به یک خطی با بعد فرکانس طیف ها نیست در نتیجه تمام اطلاعات DOA در سطح فریم جمع می شوند و این کار را برای تعیین DOA های درست برای کلاس های صدای مختلف دشوار میکند، ضمن اینکه در صورت استفاده از مقیاس خطی این مزیت را داریم که اطلاعات جهت دار در هر باند فرکانسی حفظ می شوند به همین دلیل در ویژگی جدید از مقیاس خطی استفاده میشود

بطوریکه توان سیگنال و منبع DOA یک نگاشت طیفی زمانی دقیق برقرار باشد. در این ویژگی از طیف خطی (اندازه بصورت لگاریتمی و فرکانس خطی) به همراه نرمالیزه شده بردار ویژه اصلی (بردار ویژه متناظر با بزرگترین مقدار ویژه) ماتریس همبستگی فضایی در هر بین زمان فرکانسی طیف ها استفاده می شود. طیف log-linear برای طیف های مختلط $X(t, f)$ را بصورت زیر محاسبه می کنیم که M تعداد کانال های آرایه میکروفون (با هر ساختار دلخواهی)، T تعداد فریم های زمانی و F تعداد بین های فرکانس است:

$$LINSPEC(t, f) = \log(|X(t, f)|^2) \in \mathbb{R}^{M \times T \times F} \quad (14)$$

با فرض خطی تغییرناپذیر با زمان بودن سیستم، اگر تعداد منابع صدا را برابر L در نظر بگیریم (در روابط برای بردار هدایت و... منابع را باند باریک در نظر گرفته ایم)، در این صورت بردار مشاهده شده سیگنال توسط آرایه میکروفون ها یعنی $\mathbf{x} = [x_1, \dots, x_M]^T$ بصورت زیر خواهد بود:

$$\mathbf{x}(\tau) = \sum_{i=1}^L s_i(\tau) \star \mathbf{h}(\tau, \phi_i, \theta_i) + \mathbf{n}(\tau) \quad (15)$$

که $\mathbf{h}(\tau, \phi_i, \theta_i)$ با ابعاد $M \times 1$ بردار هدایت (steering vector) در حوزه زمان متناظر با زاویه ورود برای منبع i ام است که در آن ϕ_i و θ_i به ترتیب بیانگر زوایای azimuth و elevation می باشند که میتوانند با زمان متغیر باشند، توجه شود که بردار هدایت نشاندهنده تاخیرهای فازی است (نسبت به یک نقطه مرجع) که سیگنال برای رسیدن به سنسورها خواهد داشت. نمایش STFT رابطه بالا بصورت زیر خواهد بود:

$$\mathbf{X}(t, f) = \sum_{i=1}^L S_i(t, f) \mathbf{h}(f, \phi_i, \theta_i) + \mathbf{N}(t, f) \in \mathbb{C}^M \quad (16)$$

که t و f بیانگر اندیس های زمان و فرکانس، S_i سیگنال منبع i ام و $\mathbf{h}(f, \phi_i, \theta_i)$ بردار هدایت در حوزه فرکانس و $\mathbf{N}$ نیز بردار نویز می باشد ($\mathbf{S}(t, f) = [N_1(t, f), \dots, N_L(t, f)]^T$). اگر در نظر بگیریم $\mathbf{H} = [\mathbf{h}(f, \phi_1, \theta_1), \dots, \mathbf{h}(f, \phi_L, \theta_L)] \in \mathbb{C}^{M \times L}$ و $\mathbf{S}(t, f) = [S_1(t, f), \dots, S_L(t, f)]^T$ در این صورت حاصل بالا را بصورت زیر نیز میتوان نمایش داد:

$$\mathbf{X}(t, f) = \mathbf{H}\mathbf{S}(t, f) + \mathbf{N}(t, f) \quad (17)$$

با فرض اینکه سیگنال ها و نویزها دارای میانگین صفر هستند و ناهمبسته باشند، ماتریس همبستگی $\mathbf{R}(t, f) \in \mathbb{C}^{L \times L}$ ترکیب خطی از ضرب خارجی بردارهای هدایت که دارای ضریبی هستند، خواهد بود که این ضریب توان سیگنال i ام در بین (bin) زمان فرکانسی (t, f) یعنی $\sigma_i^2(t, f)$ می باشد، بنابراین:

$$\begin{aligned}
 \mathbf{R}(t, f) &= \mathbb{E}[\mathbf{X}(t, f)\mathbf{X}^H(t, f)] = \mathbf{H}\mathbb{E}[\mathbf{S}(t, f)\mathbf{S}^H(t, f)]\mathbf{H}^H + \mathbb{E}[\mathbf{N}(t, f)\mathbf{N}^H(t, f)] \\
 \Rightarrow \mathbf{R}(t, f) &= \mathbf{H}\mathbf{R}_{ss}(t, f)\mathbf{H}^H + \mathbf{R}_n(t, f) \\
 &= \sum_{i=1}^L \sigma_i^2(t, f) \mathbf{h}(f, \phi_i, \theta_i) \mathbf{h}^H(f, \phi_i, \theta_i) + \mathbf{R}_n(t, f) \quad (18)
 \end{aligned}$$

چراکه باتوجه به فرض ناهمبسته بودن منابع، ماتریس $\mathbf{R}_{ss}(t, f)$ که دارای ابعاد $L \times L$ می باشد، یک ماتریس قطری با درایه های $\sigma_i^2(t, f)$ خواهد بود، بطور مشابه باتوجه به اینکه نویزهای گوسی با میانگین صفر را هم ناهمبسته در نظر گرفتیم، اگر واریانس همه آنها را یکسان، بطور مثال برابر σ^2 در نظر بگیریم $\mathbf{R}_n(t, f)$ برابر $\sigma^2 \mathbf{I}$ می شود که $\mathbf{I}$ ماتریس همانی با ابعاد $L \times L$ است؛ همچنین باید در نظر داشت که اگرچه انعکاس ها هم در بردار نویز در نظر گرفته شده اند اما فرض ناهمبسته بودن نویزها را میتوان همچنان سطح انعکاس به اندازه کافی پایین باشد، در نظر گرفت. توجه شود در روابط نوشته شده از این نکات نیز استفاده شد که $(\mathbf{H}\mathbf{S}(t, f))^H = \mathbf{S}^H(t, f)\mathbf{H}^H$ و $\mathbf{H}\mathbf{R}_{ss}(t, f) = [\sigma_1^2 \mathbf{h}(f, \phi_1, \theta_1), \dots, \sigma_L^2 \mathbf{h}(f, \phi_L, \theta_L)]$ همچنین در عمل برای محاسبه کوواریانس میتوان از تقریب آن استفاده کرد که میتوان آن را بصورت $\hat{\mathbf{R}}(t, f) = \frac{1}{2T_r+1} \sum_{\tau=-T_r}^{T_r} \mathbf{X}(t+\tau, f)\mathbf{X}^H(t+\tau, f)$ بدست آورد که $2T_r+1$ سائز پنجره است. باتوجه به رابطه (۱۸) در بلوک های زمان فرکانسی که یک منبع وجود دارد، یا به عبارتی فقط یک منبع بر دیگر منابع و انعکاس ها غالب باشد (مثلا منبع i ام) در این صورت در این زمان و فرکانس، برای ماتریس همبستگی میتوان از تقریب زیر استفاده کرد :

$$\hat{\mathbf{R}}(t, f) = \sigma_i^2(t, f) \mathbf{h}(f, \phi_i, \theta_i) \mathbf{h}^H(f, \phi_i, \theta_i) \quad (19)$$

باتوجه به اینکه گفتیم بردار هدایت شامل تاخیر فاز ها میشود بنابراین میتوان دید که حاصل $\mathbf{h}^H(f, \phi_i, \theta_i) \mathbf{h}(f, \phi_i, \theta_i)$ برابر یک می شود، بنابراین با ضرب عبارت بالا از سمت راست در $\mathbf{h}(f, \phi_i, \theta_i)$ خواهیم داشت :

$$\hat{\mathbf{R}}(t, f) \mathbf{h}(f, \phi_i, \theta_i) = \sigma_i^2(t, f) \mathbf{h}(f, \phi_i, \theta_i) \quad (20)$$

بنابراین با توجه به این رابطه، میتوان تقریبی از $\mathbf{h}(f, \phi_i, \theta_i)$ را برحسب بردار ویژه اصلی بدست آورد. توجه شود که چون ماتریس همبستگی ماتریس هرمیتی است و باتوجه فرم $\mathbf{R}(t, f) = \mathbf{H}\mathbf{R}_{ss}(t, f)\mathbf{H}^H$ که در روابط قبلی هم نوشتیم میتوان به این نتیجه رسید، چراکه میدانیم اگر $\mathbf{v}$ بردار ویژه ماتریس همبستگی باشد $(\mathbf{R}(t, f)\mathbf{v} = \lambda \mathbf{v})$ با تجمیع این بردار ویژه ها در ستون های ماتریس $\mathbf{V}$ و مقادیر ویژه در ماتریس قطری Λ ، خواهیم داشت $\mathbf{R}(t, f)\mathbf{V} = \Lambda\mathbf{V}$ که درنهایت با ضرب از سمت راست در وارون ماتریس $\mathbf{V}$ به فرم مشابه بالا میرسیم و چون برای حالتی که در نظر گرفتیم ماتریس همبستگی مرتبه یک بوده و فقط یک مقدار ویژه

غیر صفر دارد ($\mathbf{R}_{ss}(t, f)$ فقط شامل یک درایه غیر صفر بزرگترین مقدار از بین σ_i^2 ها است) و در $\mathbf{H}$ هم فقط ستون غیر صفر متناظر با این درایه یعنی $\mathbf{h}(f, \phi_i, \theta_i)$ را خواهیم داشت که باتوجه به توضیحاتی که ذکر شد متناظر با بردار ویژه اصلی خواهد بود. بنابراین باتوجه به مطالب گفته شده، میتوان در این بلوک ها با کمک بردار ویژه های اصلی، اطلاعاتی را در مورد جهت ها و زوایای ورود بدست آوریم. حال سعی میکنیم این بردار ویژه ها را بگونه ای نرمالیزه کنیم تا این اطلاعات را در اختیار ما قرار دهند. در میدان دور یک آرایه میکروفون ها اطلاعات مربوط به جهت ها بصورت IPD (نسبت فاز های بین دو کانال $\frac{x_i(t, f)}{x_j(t, f)}$) کدگذاری می شوند. در میدان دور یک آرایه M کاناله با هر ساختار دلخواهی، درایه m ام بردار هدایت برابر می شود با:

$$h_m(t, f, \phi, \theta) = e^{-j2\pi f \tau_{1m}} = e^{-j2\pi f \frac{d_{1m}(\phi(t), \theta(t))}{c}} \quad (21)$$

که $c \simeq 343 \frac{m}{s}$ سرعت صوت است و $d_{1m}(\phi(t), \theta(t))$ فاصله ورود طی شده (برحسب متر) بین میکروفون m و میکروفون مرجع ($m=1$) است و τ_{1m} هم اختلاف زمانی ورود (TDOA) طی شده بین میکروفون m و میکروفون اول است (باتوجه به اینکه میکروفون اول را بعنوان نقطه مرجع در نظر گرفته ایم بنابراین درایه اول بردار هدایت برابر یک خواهد بود) که برابر $\tau_{1m}(\phi(t), \theta(t)) = \frac{d_{1m}(\phi(t), \theta(t))}{c}$ می باشد. البته در شبیه سازی دقت شود به علت گسسته بودن، بجای f از $\frac{k}{N}$ استفاده میکنیم که k مقادیر از صفر تا $N-1$ را میگیرد و N تعداد نقاط برای تبدیل FFT است و این حاصل را در فرکانس نمونه برداری هم ضرب میکنیم. توجه شود که بصورت تئوری فاصله ورود بصورت زیر بدست می آید:

$$d_{1m}(\phi(t), \theta(t)) = (\zeta_1 - \zeta_m)^T \begin{bmatrix} \cos(\phi(t)) \cos(\theta(t)) \\ \sin(\phi(t)) \cos(\theta(t)) \\ \sin(\theta(t)) \end{bmatrix} \in \mathbb{R} \quad (22)$$

که ζ_1 و ζ_m مختصات کارتزین نقطه مرجع و میکروفون m ام می باشند. اطلاعات مربوط به جهت ها را میتوان به دو صورت RDOA و TDOA نمایش داد، که در اینجا ما این اطلاعات را به صورت RDOA استخراج می کنیم، یک مزیت این کار این است که دیگر نیازی نیست تا مختصات دقیق هر کدام از میکروفون ها را بدانیم چراکه اطلاعات فضایی میکروفون ها بصورت ضمنی در RDOA کدگذاری شده اند. بنابراین به محاسبه بردار فاز برپایه بردار ویژه (EPV) با هدف تخمین $[d_{12}, \dots, d_{1M}]^T$ از بردار ویژه اصلی $\mathbf{U}(t, f)$ (دارای ابعاد $M \times 1$) میپردازیم؛ بدین صورت که در ابتدا $\mathbf{U}$ را بر حسب درایه اول آن نرمالیزه میکنیم چراکه نقطه مرجع دلخواه را میکروفون اول در نظر گرفته ایم. سپس درایه اول را حذف میکنیم تا به $\bar{\mathbf{U}}$ برسیم. با برابر قرار دادن درایه های این بردار با درایه متناظر از بردار هدایت باتوجه به توضیحات و روابط ذکر شده، بصورت EPV $\tilde{\mathbf{U}} = \mathbf{V}(t, f) = -\frac{c}{2\pi f} \angle \bar{\mathbf{U}}$ بدست می آید. بنابراین برای ویژگی ورودی ها، برای یک

میکروفون M کاناله، میتوانیم از M طیف بدست آمده را در کنار $M-1$ ویژگی EPV استفاده کنیم. باتوجه به اینکه در EPV، درایه ها را با درایه اول نرمالیزه کردیم، و چون در نهایت فاز آن را در نظر گرفتیم، چون فقط فاز برای ما اهمیت خواهد داشت، در پیاده سازی بجای تقسیم بر درایه اول میتوان بقیه درایه ها را در مزدوج درایه اول ضرب کرد، بنابراین میتوان نوشت:

$$V(t, f) = -\frac{c}{2\pi f} \arg[U_1^*(t, f)U_{2:M}(t, f)] \in \mathbb{R}^{M-1} \quad (23)$$

باتوجه به ساختار معماری برای سیستم استفاده میکنیم که ذکر شد نیاز به محاسبات کمتری دارد و دارای سرعت بیشتری است (توضیحات بیشتر در قسمت بعدی)، هدف در این پژوهش افزایش سرعت در حین حفظ کارایی است، اما ویژگی بدست آمده در بالا به دلیل نیاز به محاسبه بردار ویژه های اصلی، از نظر محاسباتی پرهزینه می باشد، به همین دلیل برای اینکه ویژگی های ورودی هم با هدف سیستم در یک راستا باشند، سعی میکنیم از نسخه بهبود یافته ویژگی بالا استفاده کنیم که از نظر محاسباتی دارای کارایی بیشتری باشد. در این ویژگی بجای EPV و بردار ویژه اصلی از نسخه در فرکانس نرمالیزه شده IPD استفاده میکنیم (NIPD) [7]. از IPD باتوجه به اینکه نسبت فاز سیگنال دریافتی در دو سنسور را به ما میدهد (درنتیجه اطلاعاتی را مورد تاخیر و جهت ها ... به ما می دهد و شامل اطلاعات ضمنی و مشخصی از مکان ها با استفاده از اطلاعات فضایی است) به همین دلیل در خیلی از کاربردهای جداسازی کور منابع، تخمین DOA و.. از آنها استفاده می شود. البته در عمل این اطلاعات فضایی در IPD ها به دلیل وجود انعکاس ها و عدم داشتن تطابق بین پاسخ ها فاز بین میکروفون ها و... اغلب نویزی هستند به همین دلیل برای بهبود کارایی از شبکه های عصبی استفاده می کنیم. ویژگی NIPD را بصورت زیر محاسبه می کنیم: (باتوجه به توضیحات داده شده درایه i ام برابر نرمالیزه شده نسبت فاز کانال i ام به کانال مرجع است)

$$\Lambda(t, f) = -\frac{c}{2\pi f} \arg[X_1^*(t, f)X_{2:M}(t, f)] \in \mathbb{R}^{M-1} \quad (24)$$

برای درک معقول بودن این ویژگی، حالتی را در نظر میگیریم به در بلوک زمان فرکانس، یک منبع غالب $S(t, f)$ داشته باشیم، در این حالت:

$$X(t, f) \simeq \mathbf{h}(t, f, \phi, \theta)S(t, f) + \mathbf{N}(t, f) \quad (25)$$

در این صورت اگر RDOA در لحظه t بین کانال m و کانال مرجع را به اختصار بجای $d_{1m}(\phi(t), \theta(t))$ با $d_{1m}(t)$ نمایش دهیم؛ اگر از هرگونه اعوجاج فازی به دلیل نویز صرف نظر کنیم، باتوجه به رابطه نوشته شده، برای تخمین $\arg[X_1^*(t, f)X_{2:M}(t, f)]$ میتوان از

$\arg[h_1^*(t, f)h_{2:M}(t, f)]$ استفاده کرد (توجه شود که چون نرمالیزه کرده ایم و $S(t, f)$ در یک بلوک زمان فرکانس برای همه زوایای ورود یکسان است که به همین دلیل هم در رابطه نوشته شده آن را به فرم برداری ننوشتیم و تاثیر تغییرات سیگنال دریافتی را در بردار هدایت میبینیم، در نتیجه در رابطه نهایی نوشته شده بدلیل ضرب آن در مزدوجش و گرفتن فاز حاصل، ساده شده است)، که بدلیل ضابطه $h_m(t, f, \phi, \theta)$ که در صفحات قبل نوشتیم، خواهیم داشت :

$$\mathbf{A}(t, f) \simeq -\frac{c}{2\pi f} \arg[h_1^*(t, f)h_{2:M}(t, f)] \simeq [d_{12}(t), \dots, d_{1M}(t)]^T \quad (26)$$

بنابراین میتوان دید که به شیوه مشابه $\mathbf{V}(t, f)$ ، هم RDOA را تخمین می زند، همچنین هم ترازی دقیق زمان فرکانسی بین طیف های چندکاناله و NIPD در اسکیل فرکانسی خطی وجود خواهد داشت که در مواقعی که منابع صدا دارای همپوشانی زمانی باشند، این ویژگی ضروری خواهد بود. باتوجه به رابطه بدست آمده میتوان دید که دیگر نیازی به محاسبه U از طریق تجزیه svd ماتریس $\mathbf{X}$ نداریم و زمان انجام محاسبات بسیار کاهش می یابد، حتی در مقایسه با ویژگی GCC-PHAT در کنار طیف log-mel هم، محاسبه این ویژگی سریعتر می باشد

۳-۲- معماری MobileNet

همانطور که در مقدمه نیز ذکر شد، در سیستم این پژوهش با هدف کاهش تعداد پارامتر ها و عملیات های جمع و ضرب و زمان یادگیری نسبت به ساختارهای مرسوم CNN و ResNet و... از ساختار MobileNet استفاده می کنیم. کلیت این ساختار مشابه CNN است با این تفاوت که در اینجا بجای استفاده از کانولوشن های عادی، از کانولوشن های تفکیک پذیر در عمق (dws) استفاده میکنیم، در این حالت کانولوشن استاندارد را به دو قسمت کانولوشن عمقی و کانولوشن $1*1$ (نقطه ای) تقسیم میکنیم؛ کانولوشن عمقی تنها یک فیلتر یکسان را به تمام کانال های ورودی اعمال می کند، سپس کانولوشن نقطه ای اعمال می شود تا خروجی های حاصل از کانولوشن عمقی را با هم ترکیب کند. در کانولوشن استاندارد هم میدانیم که دو مرحله فیلتر و ادغام وجود دارد اما این دو مرحله در کانولوشن استاندارد در یک مرحله انجام می شوند، یعنی هر کانال با یک فیلتر مختص (کانال متناظر با آن در فیلتر چندکاناله) به خودش کانالو می شود و سپس خروجی های بدست آمده که به تعداد کانال ها هستند، با هم جمع میشوند تا به یک خروجی برسیم، این کار به تعداد فیلتر و کرنلی که داشته باشیم انجام میشود تا در نهایت تعداد خروجی ها برابر تعداد

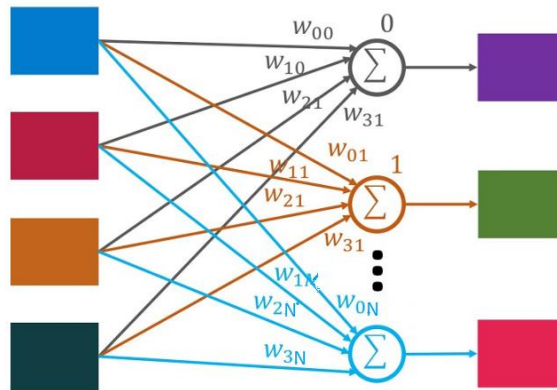
فیلترها بدست می آید. بنابراین در کانولوشن استاندارد تنها در ابتدا کانولوشن وجود دارد و مرحله ادغام یک جمع ساده است اما در کانولوشن dws هر دو مرحله شامل کانولوشن می شوند، به عبارتی عملیات قبلی به دو لایه تقسیم می شود، یک لایه جداگانه برای فیلتر کردن (کانولوشن عمقی) و یک لایه جداگانه برای ترکیب کردن (کانولوشن نقطه ای). پس همانطور که گفته شد در کانولوشن استاندارد لایه بطور مثال ورودی فیچر مپ F با ابعاد $D_F \times D_F \times M$ را میگیرد و فیچر مپ G با ابعاد $D_G \times D_G \times N$ را ایجاد میکند (ورودی بطور مثال با طول و عرض یکسان D_F در نظر گرفته شده، طول و عرض خروجی هم فرض میکنیم مشابه ورودی باشند)، M تعداد کانال ورودی (عمق ورودی) و N تعداد کانال های خروجی (عمق خروجی) است. لایه کانولوشن استاندارد توسط کرنل کانولوشن K با ابعاد $D_K \times D_K \times M \times N$ معرفی می شود. در این حالت خروجی با فرض stride برابر یک و padding بصورت زیر خواهد بود :

$$G_{k,l,n} = \sum_{i,j,m} K_{i,j,m,n} \cdot F_{k+i-1,l+j-1,m} \quad (27)$$

بنابراین میتوان گفت که کانولوشن استاندارد دارای هزینه محاسباتی $D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F$ خواهد بود. در کانولوشن dws باتوجه به کاری که انجام می شود ارتباط بین تعداد کانال های خروجی و سایز کرنل از بین می رود.

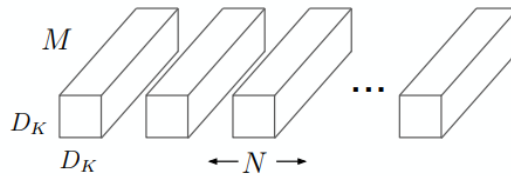
همانطور که گفته شد در کانولوشن dws، کانولوشن استاندارد (عمل فیلتر و ادغام) را به دو قسمت کانولوشن عمقی و کانولوشن نقطه ای تقسیم میکنیم. در کانولوشن عمقی برخلاف کانولوشن استاندارد که N کرنل داشتیم، فقط یک کرنل (مشخصا با عمق مشابه ورودی) خواهیم داشت بدین ترتیب بر روی هر کانال ورودی فقط یک فیلتر اعمال می شود. مرحله ادغام در کانولوشن استاندارد یک جمع ساده است اما در کانولوشن dws از کانولوشن 1×1 استفاده می شود؛ این کانولوشن 1×1 نقطه ای خروجی های کانولوشن عمقی مرحله قبل را با هم ادغام می کند (در مرحله قبل بجای تعریف N کرنل، از یک کرنل استفاده کردیم اما در این مرحله از N کرنل 1×1 با عمق M استفاده می کنیم)، بنابراین در این حالت N ترکیب خطی مختلف از خروجی های لایه قبل خواهیم داشت (جمع وزن دار صفحات)، چراکه هر کرنل 1×1 دارای M وزن است که به هر صفحه از فیچر مپ ورودی یک وزن می دهد و آن ها را با هم جمع میکند، بنابراین به نوعی میتوان نقش آن را مشابه نوروں ها دانست یعنی کانولوشن 1×1 مشابه یک لایه fully connected است که N نوروں دارد و هر ورودی اش معادل با یک صفحه است.¹

¹ <https://howsam.org/mobilenet/>

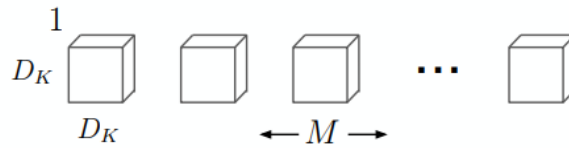


شکل (۲-۴) N فیلتر در نقش یک لایه fully connected. هر نورون معادل یک فیلتر 1×1

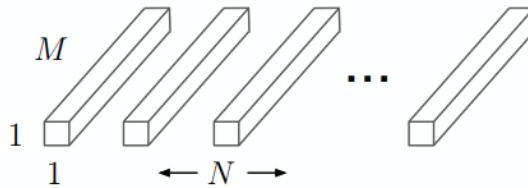
بنابراین شکل کلی کانولوشن ها را میتوان بصورت زیر نمایش داد:



شکل (۲-۵) فیلترهای کانولوشن استاندارد



شکل (۲-۶) فیلترهای کانولوشن عمقی



شکل (۲-۷) فیلترهای کانولوشن نقطه ای

در این معماری از batchnorm و تابع غیرخطی ReLU در هر دو لایه استفاده می شود. کانولوشن عمقی با یک فیلتر به ازای هر کانال را میتوان بصورت زیر نوشت:

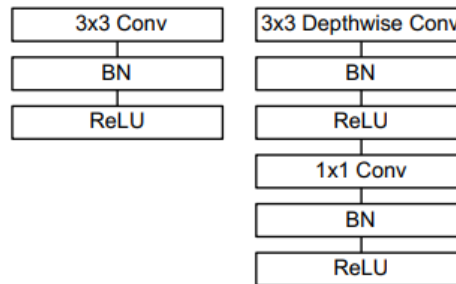
$$\hat{G}_{k,l,n} = \sum_{i,j} \hat{K}_{i,j,m} \cdot F_{k+i-1,l+j-1,m} \quad (28)$$

که $\hat{K}$ کرنل کانولوشن عمقی با ابعاد $D_K \times D_K \times M$ است که m امین فیلتر آن به کانال m ام F اعمال می شود تا کانال m ام خروجی $\hat{G}$ ایجاد شود. بنابراین کانولوشن عمقی دارای هزینه محاسباتی

$D_K \cdot D_K \cdot M \cdot D_F \cdot D_F$ است که نسبت به کانولوشن استاندارد بسیار کاراتر می باشد اگرچه فقط کانال های ورودی را فیلتر و میکند و آنها را ادغام نمیکند تا ویژگی های جدیدی درست شوند، که این کار را همانطور که گفته شد در لایه دوم انجام میدهم؛ بنابراین هزینه محاسباتی کلی کانولوشن dws، مجموع این دو مرحله خواهد بود که برابر $D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F$ می شود. بنابراین هزینه محاسباتی به مقدار زیر کاهش می یابد:

$$\frac{D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F}{D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F} = \frac{1}{N} + \frac{1}{D_K^2} \quad (29)$$

که باتوجه به اینکه در این ساختار ما از کانولوشن عمقی 3×3 استفاده می کنیم، هزینه محاسباتی در حدود 8 تا 9 برابر کمتر از کانولوشن استاندارد خواهد شد درحالیکه دقت هم به میزان بسیار کمتری کاهش خواهد یافت.



شکل (۸-۲) کانولوشن dws (شکل سمت راست) و استاندارد. البته در سیستم ما بعد از اولین کانولوشن ها از average polling نیز استفاده شده است

فصل ۳

فصل ۳: شبیه سازی

فصل سوم در برگیرنده توضیحات مربوط به ساختار سیستم و شبیه سازی آن و بررسی نتایج بدست آمده است.

۱-۳-مقدمه

برای شبیه سازی و فرآیند آموزش و ارزیابی از دیتاست زیر استفاده شده است^۱:

TAU Spatial Sound Events 2019 – Ambisonic and Microphone Array, Development Datasets
این مجموعه شامل دو دیتاست می شود که ما در اینجا از دیتاست مربوط به Microphone Array استفاده می کنیم که در آن موارد ضبط شده توسط یک میکروفون چهارکاناله جهت دار با پیکربندی آرایه ای چهاروجهی (tetrahedral) فراهم شده اند. در موارد ضبط شده، از منابع صدای ایستان و یازده کلاس مختلف صدا استفاده شده است که در هریک زمان شروع و پایان و زوایای ورود مشخص شده است. مجموعه شامل 400 صدای ضبط شده با مدت یک دقیقه می باشد (با فرکانس نمونه برداری 48 کیلوهرتز). این 400 فایل به چهار قسمت 100 تایی تقسیم می شوند، که از دو قسمت برای آموزش، یک قسمت برای validation و یک قسمت برای ارزیابی استفاده می کنیم. این فایل ها توسط پاسخ های ضربه (IRs) جمع آوری شده در پنج مکان داخلی در 504 ترکیب یکتا زوایای azimuth و elevation، سنتز شده اند. همچنین این IRs جمع آوری شده با رویدادهای صوتی ایزوله شده کانالو شده و برای هرچه بیشتر به واقعیت نزدیک شدن، نویز طبیعی محیط هم به آنها اضافه شده است بطوریکه SNR متوسط صداها برابر 30dB باشد. همچنین برای ایجاد چالش بیشتر رویدادهای صدا ممکن است با هم همپوشانی داشته باشند (حداکثر دو رویداد). در شبیه سازی برای انجام سریع محاسبات هم از مدل های pytorch استفاده شده و محاسبات بر روی کارت گرافیک (با استفاده از cuda) انجام می شوند.

۲-۳-شبیه سازی ویژگی های ورودی

در استخراج ویژگی ها هرچه فرکانس نمونه برداری، تعداد نقاط FFT، بازه فرکانسی، تعداد بلوک ها بیشتر باشد، دقت بیشتری خواهیم داشت، اما در اینجا بدلیل محدودیت سخت افزاری و داشتن سرعت بیشتر در اجرا، فرکانس نمونه برداری را 24 کیلوهرتز، تعداد نقاط FFT را 512 در نظر میگیریم و فرکانس قطع برای طیف ها را هم 8 کیلوهرتز در نظر میگیریم. پارامتر hop length را هم برابر 20 میلی ثانیه (480 سمپل)

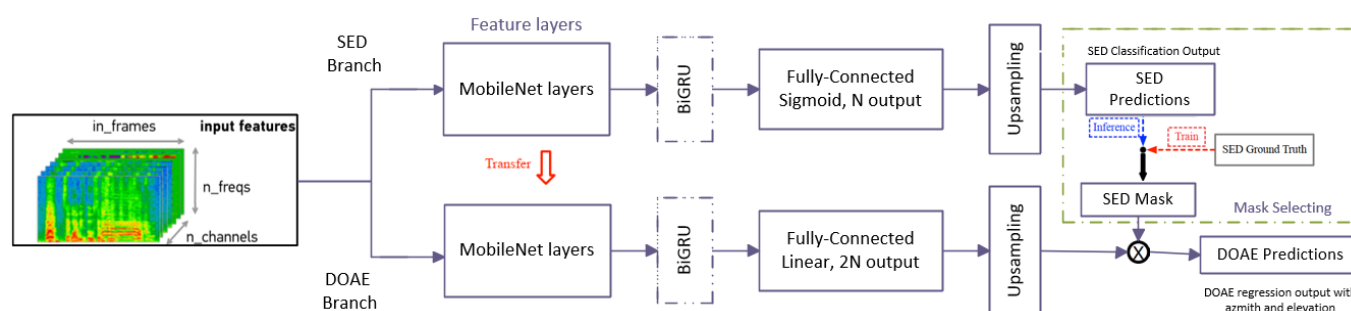
¹ <https://zenodo.org/records/2599196>

در نظر میگیریم در این صورت برای یک صوت به مدت یک دقیقه، 3000 فریم خواهیم داشت (هر چه طول کمتر باشد تعداد فریم ها بیشتر و حجم داده ها هم بیشتر می شود). تعداد بین ها نیز مشخصا برابر تعداد نقاط FFT تقسیم بر دو، بعلاوه یک خواهد شد. کدهای مربوطه در قسمت پیوست ها قرار داده شده اند.

۳-۳- پیاده سازی شبکه عصبی

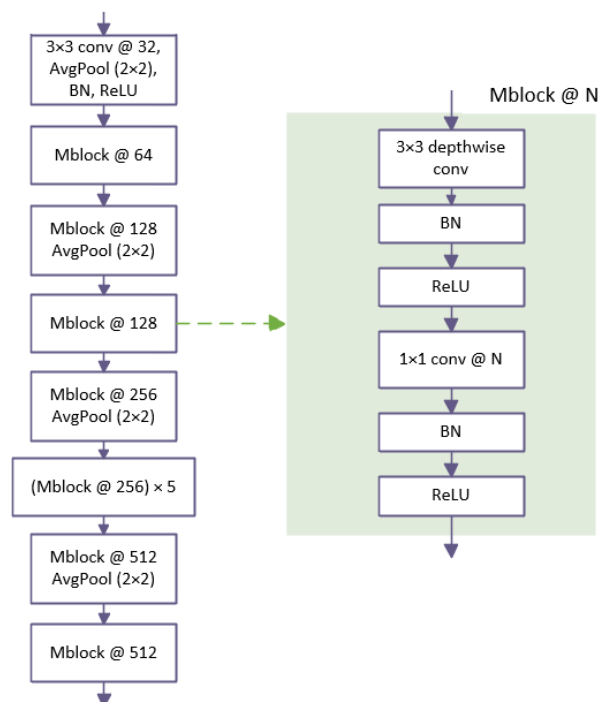
در ابتدا لازم به ذکر است که در فرآیند یادگیری از بهینه ساز adam استفاده شده و نرخ یادگیری در ابتدا برابر 0.001 قرار داده شده است و پس از طی کردن حدود سی epoch، در هر مرحله پس طی کردن حدود سه epoch، نرخ یادگیری را ده درصد کاهش می دهیم، همچنین سایز batch هم برابر 32 در نظر گرفته شده است.

باتوجه به توضیحات ذکر شده در مورد ساختار، شکل کلی آن بصورت زیر خواهد بود:



شکل (۳-۱) ساختار سیستم طراحی شده

برای ساختار MobileNet هم از ساختاری مشابه ساختار استفاده شده در [4] استفاده می کنیم، البته در آنجا تعداد کانال های خروجی 1024 در نظر گرفته شده، اما در اینجا باتوجه به کوچکتر بودن دیتاست استفاده شده، برای جلوگیری از overfit شدن سیستم، بهتر است سیستم خیلی عمیق نشود تعداد کانال های خروجی را 512 در نظر می گیریم، به همین دلیل نیاز است که ساختار را تغییر دهیم، همچنین همانطور که گفته شد در لایه اول از کانولوشن استاندارد (مشابه ساختار در شکل 2-8 به همراه avgpool) استفاده می کنیم، در نتیجه از ساختار زیر استفاده می کنیم (سمبل @ نشاندهنده تعداد فیچر مپ ها یا همان کانال های خروجی است و در MBlock هایی که AvgPool داریم، این pooling پس از کانولوشن dws انجام می شود):



شکل (۳-۲) ساختار MobileNet layers استفاده شده در شکل 3-1

در ساختار کلی شکل ۳-۱، در فرآیند آموزش در ابتدا ویژگی های استخراج شده که دارای ابعاد $C \times T \times F$ می باشند که C تعداد فیچر مپ ها، T تعداد بلوک های زمانی و F تعداد بلوک های فرکانسی به شاخه SED داده می شوند، پس از استخراج ویژگی ها توسط لایه های MobileNet به طریقی که قبلا گفته شد، باتوجه به تعداد pooling های دو در دوی استفاده شده، ابعاد خروجی برابر $C_{out} \times \frac{T}{16} \times \frac{F}{16}$ خواهد شد که C_{out} که تعداد فیچر مپ های خروجی در لایه آخر می باشد، پس از آن داده از یک لایه average pooling کلی عبور داده می شود تا بعد F را کاهش دهیم و در ادامه جای ابعاد آن را عوض میکنیم که به ابعاد $\frac{T}{16} \times C_{out}$ می رسیم. پس از آن باتوجه به ماهیت داده که داده های زمانی و بصورت دنباله می باشند، برای استفاده از وابستگی های بلند مدت در داده های ورودی و در نتیجه بهبود کارایی میتوان از biGRU استفاده کرد (در GRU مشکل از بین رفتن گرادیان ها در LSTM از بین می رود و به همین دلیل میتوان وابستگی های بلند مدت تری را حفظ کرد و با این تفاوت که هر گیت هردو عمل فراموش کردن و آپدیت کردن را انجام می دهد)، در biGRU دیتا در هردو جهت پردازش می شود بنابراین برای پردازش در زمان حال هم از اطلاعات گذشته و هم اطلاعات آینده استفاده می شود، برای مشاهده روابط و ساختار GRU و biGRU میتوان به مقاله [8] مراجعه کرد. پس از عبور از biGRU در سیستم، ابعاد خروجی بدون تغییر می مانند. در ادامه آن را به لایه fully connected می دهیم (در پیاده سازی از دو نوع با یک لایه مخفی و دو لایه مخفی استفاده شد) و سایر خروجی برابر $\frac{T}{16} \times N$ می شود که N تعداد کلاس های صدا است. پس از آن از upsampling در بعد زمانی

استفاده میکنیم تا ساینز آن ثابت و برابر T بماند. در انتها از تابع فعالسازی sigmoid استفاده میکنیم؛ پیش بینی های SED را به کمک یک حد آستانه فعالسازی (در اینجا 0.3) انتخاب می کنیم. برای خطای کلاس بندی هم از binary cross-entropy استفاده می شود.

پس از این مرحله شاخه DOAE را آموزش می دهیم. برای شروع فرآیند آموزش و برای مقادیر اولیه پارامترهای لایه های MobileNet از مقادیر بدست آمده از شاخه SED استفاده می کنیم و لایه های ویژگی تعلیم داده شده را به این شاخه منتقل میکنیم، بقیه ساختار مشابه شاخه قبل است، فقط خروجی های پس از لایه fully connected یک بردار به ابعاد 2N از مقادیر خطی خواهد بود که زوایای azimuth و elevation برای N رویداد خواهند بود. در ادامه از SED های داده مرجع بعنوان یک ماسک، برای آموزش DOAE استفاده می شود تا بتوان متوجه شد که آیا زوایای متناظر بدست آمده با کلاس ها، در حال حاضر فعال هستند یا خیر، در انتها برای خطای رگرسیون DOAE از MAE استفاده می شود. در مرحله استنتاج و تست، ابتدا شاخه SED، پیش بینی های رویدادهای صدا را تخمین زده و سپس از آنها بعنوان ماسک SED برای شاخه DOAE استفاده میشود تا در نهایت زوایای ورود تخمین زده شوند. دلیل استفاده از این ساختار دو مرحله ای هم همانطور که قبلا ذکر شد این است که برای بهبود عملکرد کاری کنیم که هر شاخه فقط مسئول یک وظیفه باشد در حالیکه شاخه DOAE هم همچنان بتواند از اطلاعات بدست آمده از شاخه SED استفاده کند. کدهای مربوطه در قسمت پیوست ها قرار داده شده اند.

۴-۳- معیارهای ارزیابی^۱

برای SED و DOAE از معیارهای جداگانه ای استفاده می کنیم. برای SED معیارهای نرخ خطا بر پایه بخش (ER) و F-score را در طول های یک ثانیه ای محاسبه میکنیم [9]. معیار ER بصورت زیر محاسبه می شود :

$$ER = \frac{\sum_{k=1}^K S(k) + \sum_{k=1}^K D(k) + \sum_{k=1}^K I(k)}{\sum_{k=1}^K N(k)} \quad (30)$$

که برای قطعه یک ثانیه ای k ام، $N(k)$ تعداد کلاس های رویدادهای صوتی فعال در مرجع است (کلاس در قطعه فعال در نظر گرفته میشود اگر در هر یک از فریم های این قسمت، فعال باشد) و داریم:

¹ Evaluation metrics

$$S(k) = \min(FN(k), FP(k)), \quad D(k) = \max(0, FN(k) - FP(k)), \\ I(k) = \max(0, FP(k) - FN(k)) \quad (31)$$

$FP(k)$ تعداد کلاس های رویدادهای صوتی است که در پیش بینی فعال اما در مرجع غیرفعال بوده اند و $FN(k)$ هم تعداد کلاس های رویدادهای صوتی است که در پیش بینی غیرفعال اما در مرجع فعال هستند. اگر تعداد کلاس های رویدادهای صوتی که هم در مرجع و هم در پیش بینی فعال بوده اند را با $TP(k)$ نمایش دهیم، معیار F-score بصورت زیر محاسبه می شود:

$$precision = \frac{\sum_{k=1}^K TP(k)}{\sum_{k=1}^K TP(k) + \sum_{k=1}^K FP(k)}, \quad recall = \frac{\sum_{k=1}^K TP(k)}{\sum_{k=1}^K TP(k) + \sum_{k=1}^K FN(k)}, \\ \rightarrow F = 2 \frac{precision \cdot recall}{precision + recall} = \frac{2 \sum_{k=1}^K TP(k)}{2 \sum_{k=1}^K TP(k) + \sum_{k=1}^K FP(k) + \sum_{k=1}^K FN(k)} \quad (32)$$

هرچه ER مقدار کمتری و F-score مقدار بیشتری داشته باشد، نشاندهنده عملکرد بهتر سیستم خواهد بود. برای DOAE از معیارهای خطای DOA و frame recall استفاده می کنیم [10]. خطای DOA میانگین خطای زاویه ای برحسب درجه بین DOA های مرجع و پیش بینی شده است. بطور مثال برای یک ورودی دارای T فریم زمانی، اگر DOA_R^t لیست تمام زوایای ورود در فریم زمانی t و DOA_E^t لیست تمام زوایای ورود تخمین زده شده باشد، خطای DOA بصورت زیر محاسبه می شود:

$$DOA_{error} = \frac{1}{\sum_{t=1}^T D_E^t} \sum_{t=1}^T H(DOA_R^t, DOA_E^t) \quad (33)$$

که D_E^t تعداد زوایای ورود تخمین زده شده در DOA_E^t در فریم زمانی t ام است (از $\sum_{t=1}^T D_E^t$ برای نرمالیزه کردن بر تمام تخمین ها استفاده میکنیم) و $H(\cdot)$ بیانگر الگوریتم Hungarian است که برای تطبیق هر زاویه ورود تخمین زده شده با متناظرش در مرجع استفاده می شود، بنابراین این الگوریتم مسئله را با تخمین زدن هزینه های بصورت جفتی برای هر زاویه ورود تخمین زده شده و مرجع با استفاده از زاویه مرکزی بینشان ($\sigma = \cos^{-1}(\sin\theta_R \sin\theta_E + \cos\theta_R \cos\theta_E \cos(|\phi_R - \phi_E|))$) حل می کند. برای فریم های زمانی که تعداد زاویه های تخمین زده شده و در مرجع یکسان نباشند از معیار دوم یعنی frame recall استفاده می کنیم که بصورت زیر محاسبه می شود که در آن D_R^t تعداد زاویه های ورود در DOA_R^t در فریم زمانی t ام است: (درصد تعداد فریم هایی که در آنها تعداد زاویه های تخمین زده شده درست باشد)

$$Frame\ recall = \frac{\sum_{t=1}^T 1(D_R^t = D_E^t)}{T} \quad (34)$$

(.) تابع indicator است و حاصل آن برابر یک میشود اگر شرط $D_R^t = D_E^t$ برقرار باشد و در غیر این صورت برابر صفر می شود. هرچه خطای DOA مقدار کمتری داشته باشد و frame recall مقدار بزرگتری

باشد، نشاندهنده عملکرد بهتر سیستم خواهد بود.¹

۵-۳- نتایج بدست آمده

خروجی های بدست آمده را برای ساختارهای زیر بررسی کرده ایم:

Model	# of parameters	SELD loss
Without GRU Fully connected with 1 hidden layer	831,457	0.474
1 layer biGRU Fully connected with 1 hidden layer	2,014,177	0.487
1 layer biGRU Fully connected with 1 hidden layer Log-mel +GCC-Phat	2,015,041	0.547
2 layer biGRU Fully connected with 2 hidden layer	3,582,433	0.464

که برای معیارهای ارزیابی تعریف شده، به نتایج زیر می رسیم:

Model	ER	F-score	DOA Error(°)	FR
Without GRU Fully connected with 1 hidden layer	0.272	0.837	14.42	0.782
1 layer biGRU Fully connected with 1 hidden layer	0.264	0.843	15.189	0.787
1 layer biGRU Fully connected with 1 hidden layer Log-mel +GCC-Phat	0.263	0.837	17.601	0.786
2 layer biGRU Fully connected with 2 hidden layer	0.279	0.841	14.286	0.774

با درنظر گرفتن همه معیار ها میتوان گفت که در حالت استفاده از biGRU با دو لایه و ساختار fully connected با دو لایه پنهان، به بهترین نتایج رسیده ایم. همچنین همانطور که در سطر دوم و سوم مدل ها که از یک سیستم یکسان با دو ویژگی ورودی متفاوت استفاده کرده ایم، نتایج با استفاده از ویژگی جدید، نسبت به حالت استفاده از طیف log-mel و GCC-Phat، بهتر می باشند. با مقایسه بهترین نتیجه بدست

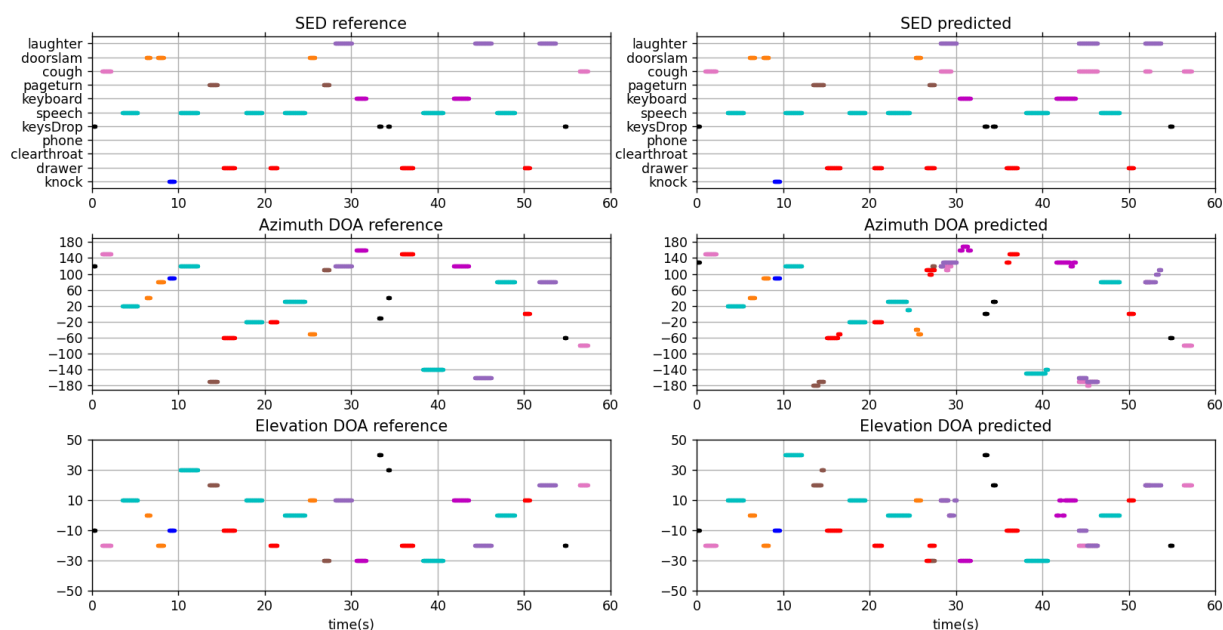
¹ <https://github.com/sharathadavanne/seld-dcase2019>

آمده با نتایج در مقالات [2] و [11] و نتایج کارگاه برگزار شده¹ dcase2019 (که در آنها از دیتاست مشابهی استفاده شده است) میتوان دید که به نتایج قابل قبولی رسیده ایم علی رغم اینکه برای ویژگی ورودی از، ویژگی های ساده تر شده ای که سریعتر محاسبه می شدند (همچنین پارامترهای برای استخراج ویژگی را هم همانطور که ذکر شد مقادیر کمتری گذاشتیم تا داده های بدست آمده حجم کمتری داشته باشند)، و در ساختار هم از معماری MobileNet استفاده کرده ایم که تعداد پارامترهای کمتری نیاز داشت و دقت آن نسبت به ساختارهای مرسوم CNN و... اندکی کمتر بود (در شبه سازی، زمان مورد نیاز برای آموزش ساختار CRNN در مقاله [2] که دارای 6 میلیون پارامتر است، حدود سه برابر زمان مورد نیاز برای ساختار MobileNet بود). بطور مثال با بررسی نتایج تیم های شرکت کننده در dcase2019 میتوان دید که نتایج بدست آمده از نتایج حدود نیمی از این تیم ها بهتر است درحالیکه در اکثر آنها از ساختارها و ویژگی های ورودی پیچیده تری استفاده شده است. در انتها با مقایسه نتایج بدست آمده میتوان دید که شاهد بهبود بسیاری نسبت به روش baseline معرفی شده برای dcase2019 هستیم [12] بطوریکه ER از 0.35 به 0.279، F-score از 0.800 به 0.841 و خطای DOA از 30.8° به 14.286° رسیده است (فقط FR افت کرده و از 0.84 به 0.774 رسیده است). در انتها لازم به ذکر است که برای رسیدن به این نتایج به تعداد epoch های کمتری هم در مقایسه با دیگر ساختارها نیاز داریم، بطوریکه برای بهترین نتیجه بدست آمده در اینجا برای شاخه SED از epoch چهار و دوم و برای شاخه DOAE از epoch شصت و هفتم استفاده شده است.

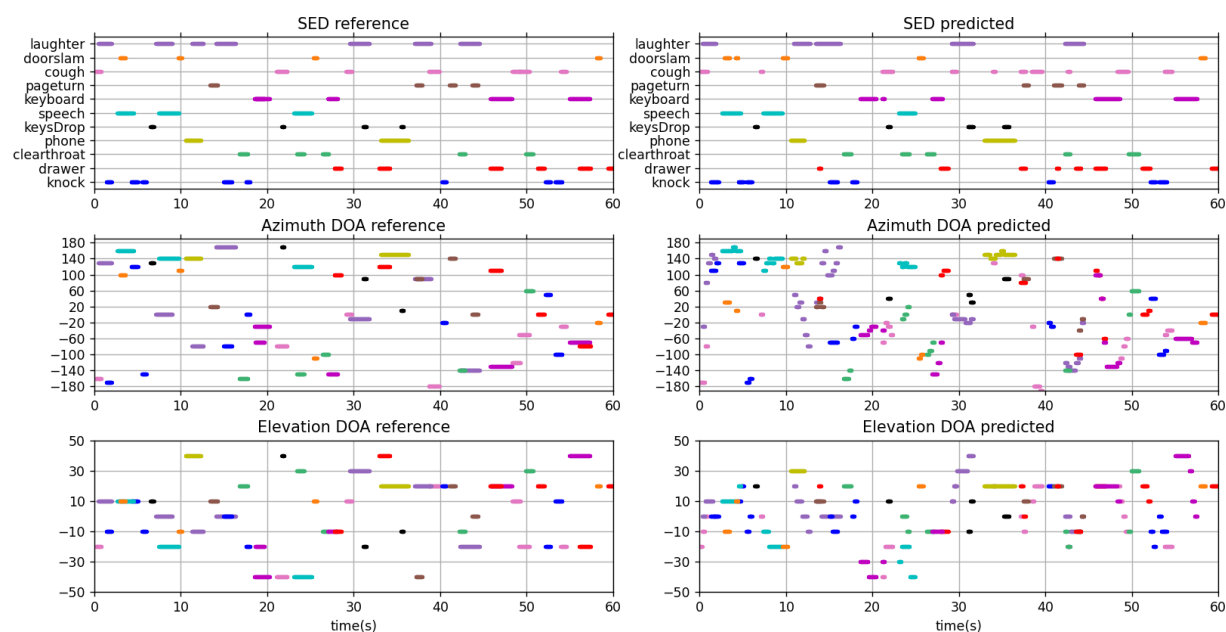
بنابراین در این پژوهش برای حل مسئله SELD سعی شد تا از یک ساختار دو مرحله ای استفاده کنیم تا کارایی در هردو بخش SED و DOAE حفظ شود که نحوه کار این سیستم در قسمت های قبلی توضیح داده شد، همچنین باتوجه به اینکه به دلیل محدودیت های سخت افزاری برای شبه سازی، هدف افزایش سرعت سیستم در عین حفظ کارایی بود از یک ویژگی ورودی جدید استفاده شد که دیدیم نسبت به ویژگی های قبلی سریعتر بدست آمده و دارای عملکرد بهتری هم هست، در همین راستا برای معماری سیستم هم از ساختار MobileNet استفاده شد که تا به حال در پژوهش دیگری از آن برای حل مساله SELD استفاده نشده است. در انتها باتوجه به اینکه گفتیم در دیتاست از پنج محیط داخلی استفاده شده است، برای هریک پیش بینی های بدست آمده را رسم می کنیم (برای هر محیط دو فایل را انتخاب کرده ایم، یکی در حالت منابع بدون همپوشانی و دیگری برای حالت منابع با همپوشانی زمانی). تصاویر اول برای حالت بدون همپوشانی و تصاویر دوم برای حالتی است که منابع دارای همپوشانی زمانی هستند.

¹ <https://dcase.community/challenge2019/task-sound-event-localization-and-detection-results>

برای محیط اول:



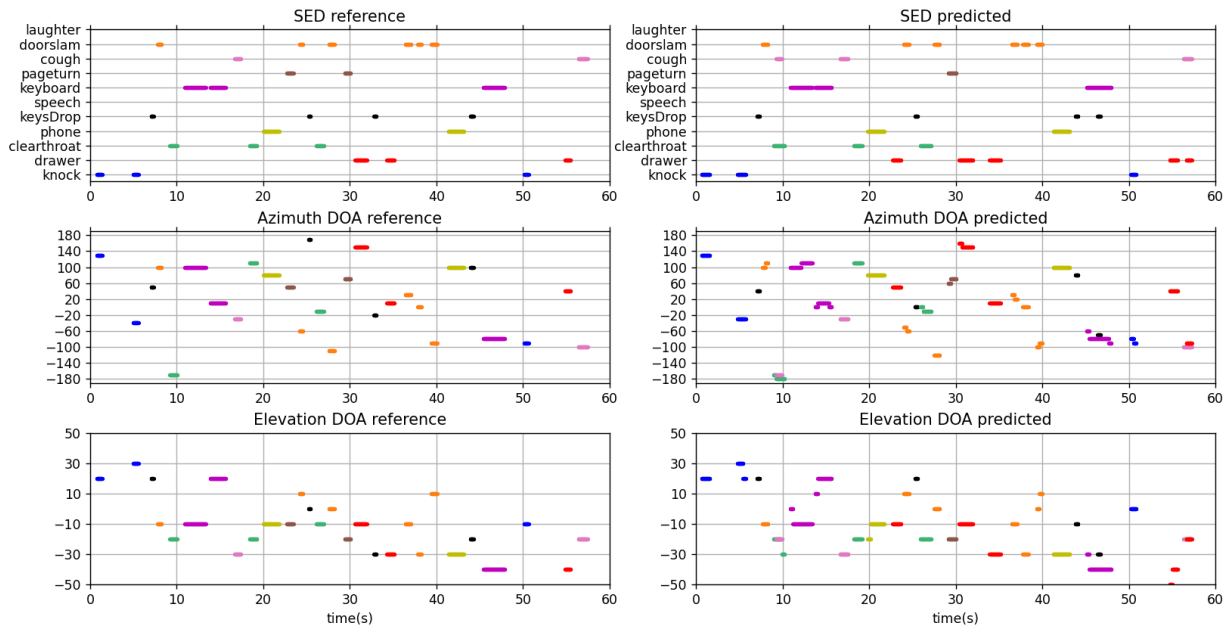
شکل (۳-۳) نتایج بدست آمده برای split1_ir0_ov1_3.wav



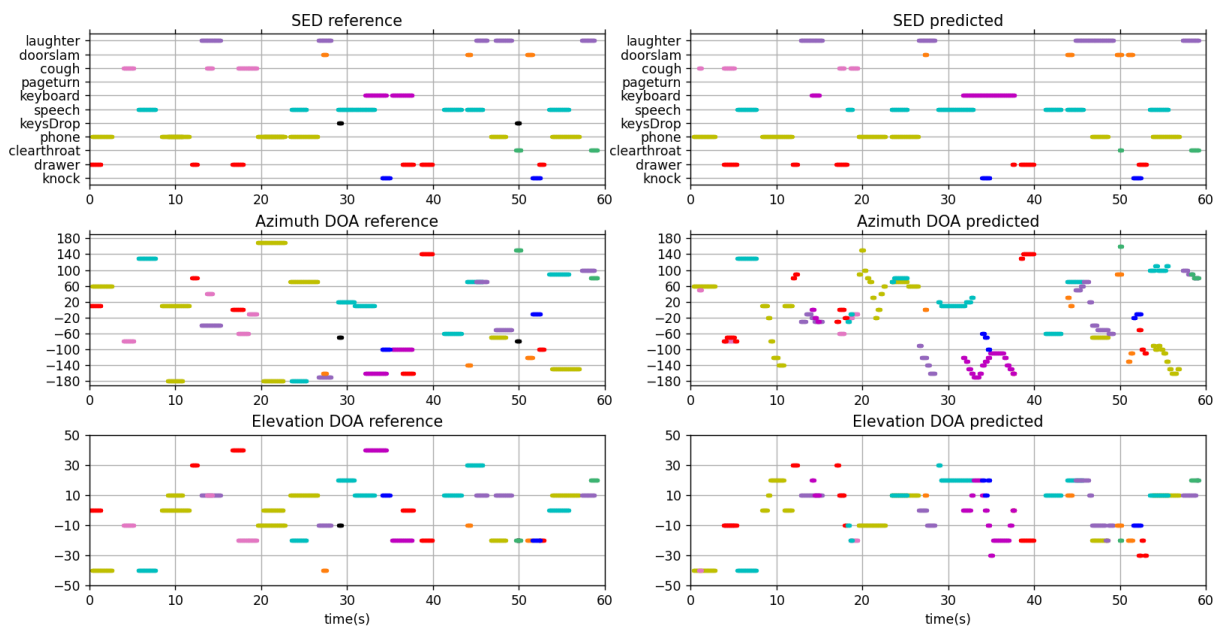
شکل (۳-۴) نتایج بدست آمده برای split1_ir0_ov2_13.wav

مطابق انتظار میتوان دید که پیش بینی های بدست آمده برای زوایای ورود، در حالتی که منابع دارای همپوشانی زمانی هستند دارای خطای بیشتری است.

برای محیط دوم:

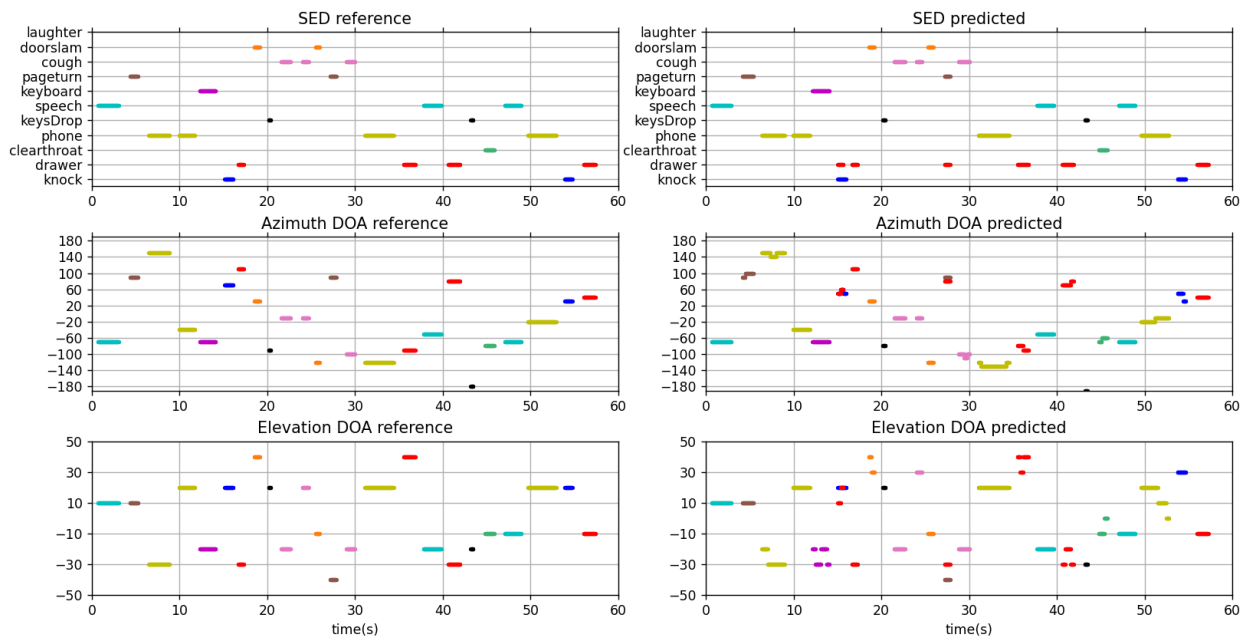


شکل (۳-۵) نتایج بدست آمده برای split1_ir1_ov1_22.wav

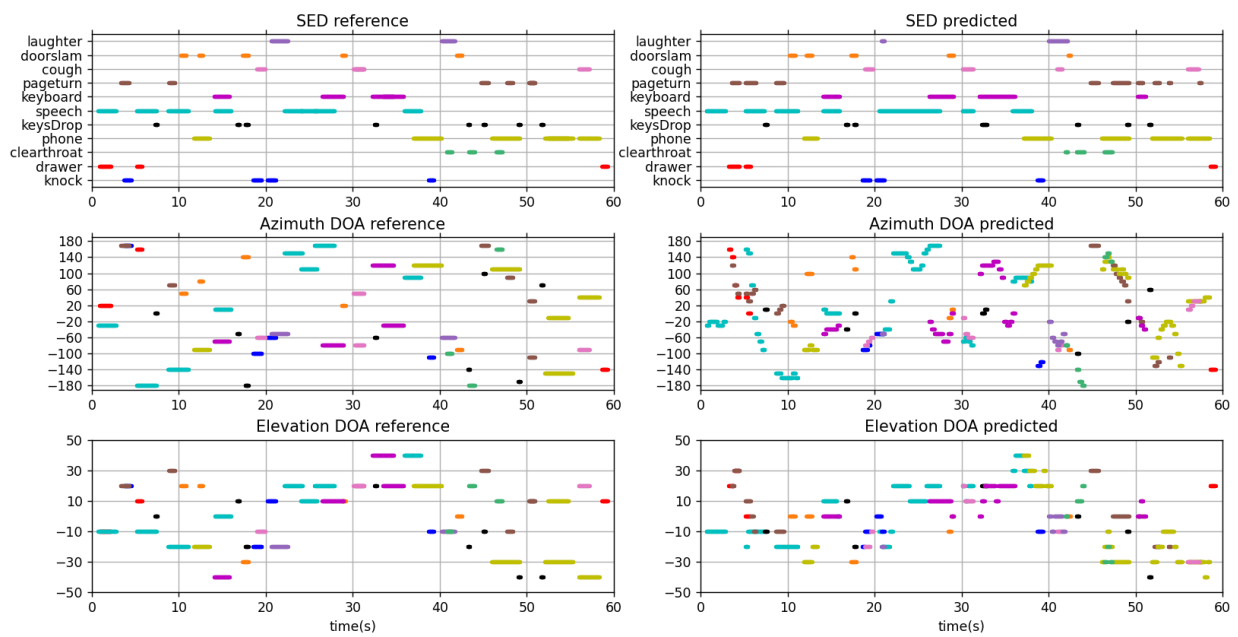


شکل (۳-۶) نتایج بدست آمده برای split1_ir1_ov2_35.wav

برای محیط سوم:

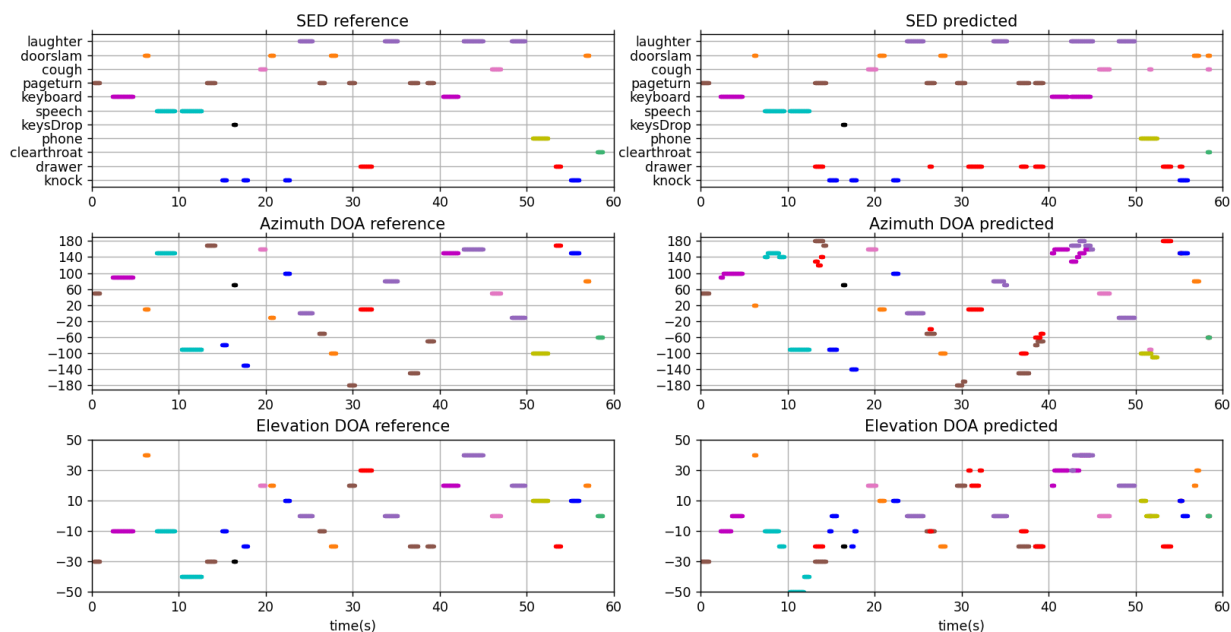


شکل (۳-۷) نتایج بدست آمده برای split1_ir2_ov1_42.wav

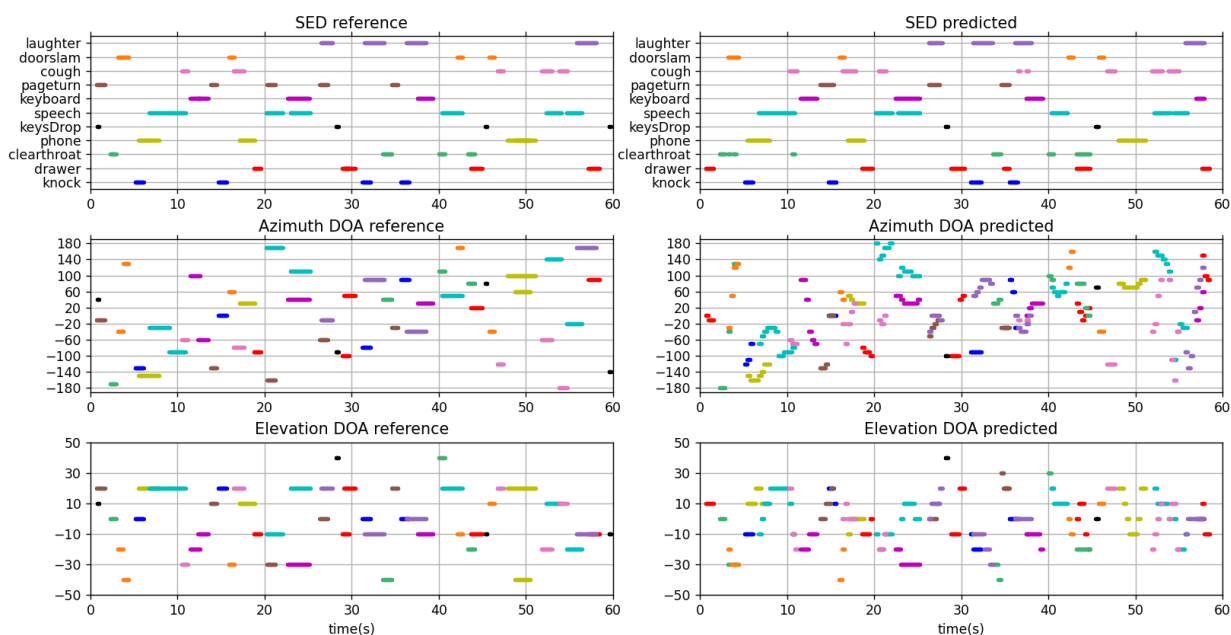


شکل (۳-۸) نتایج بدست آمده برای split1_ir2_ov2_56.wav

برای محیط چهارم:

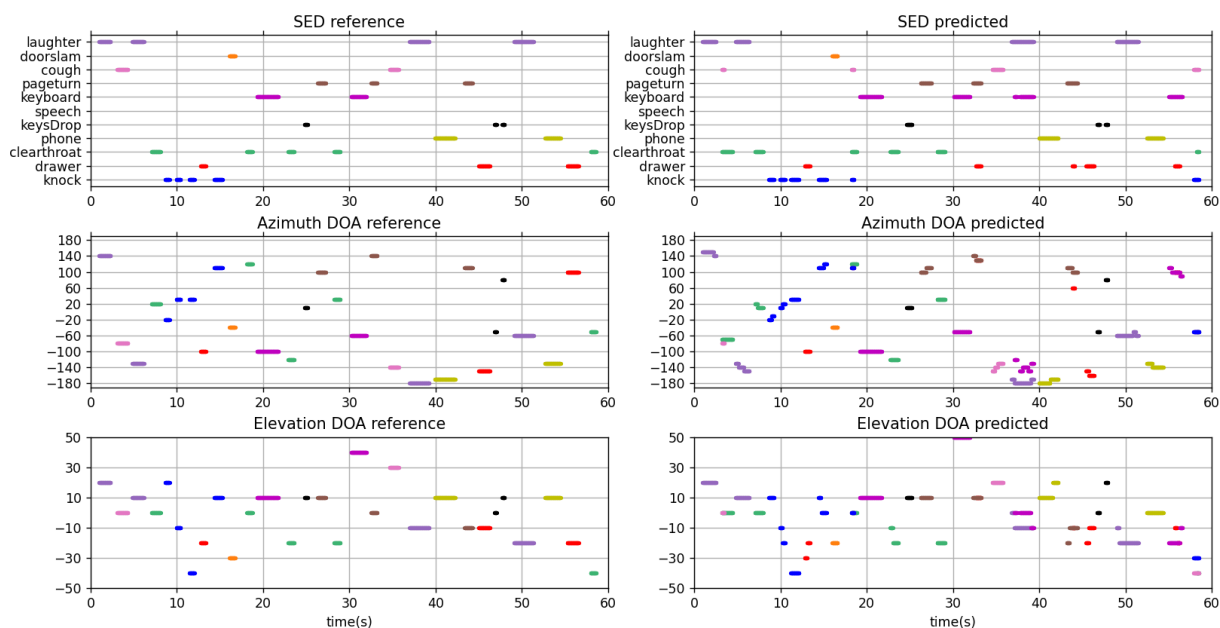


شکل (۳-۹) نتایج بدست آمده برای split1_ir3_ov1_63.wav

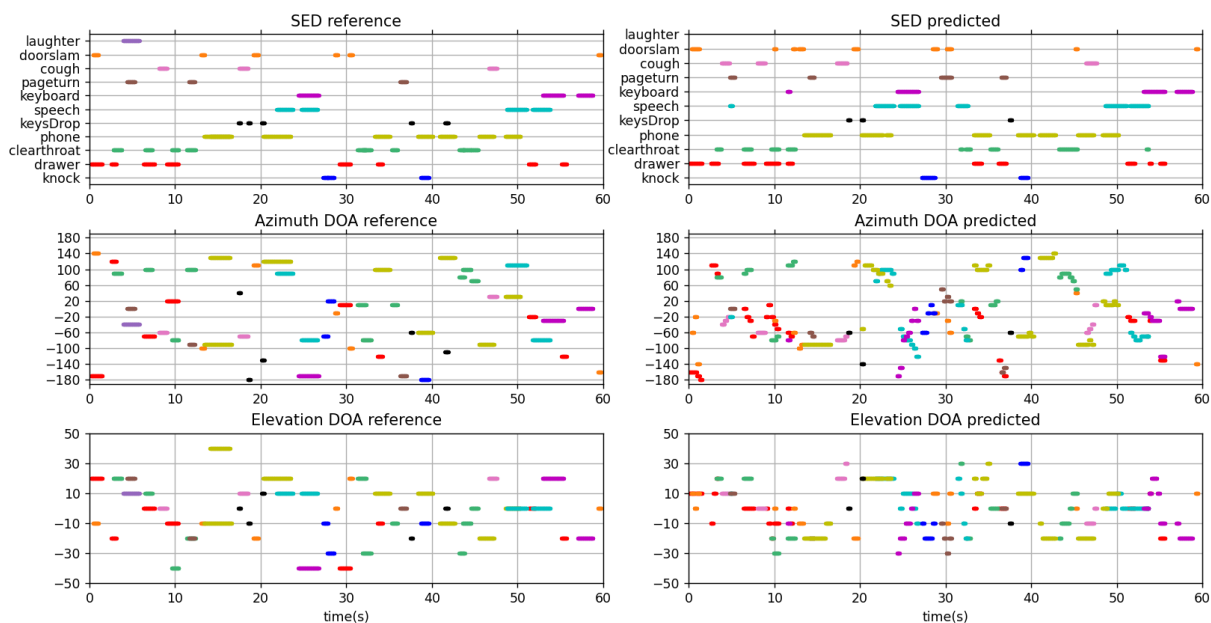


شکل (۳-۱۰) نتایج بدست آمده برای split1_ir3_ov2_74.wav

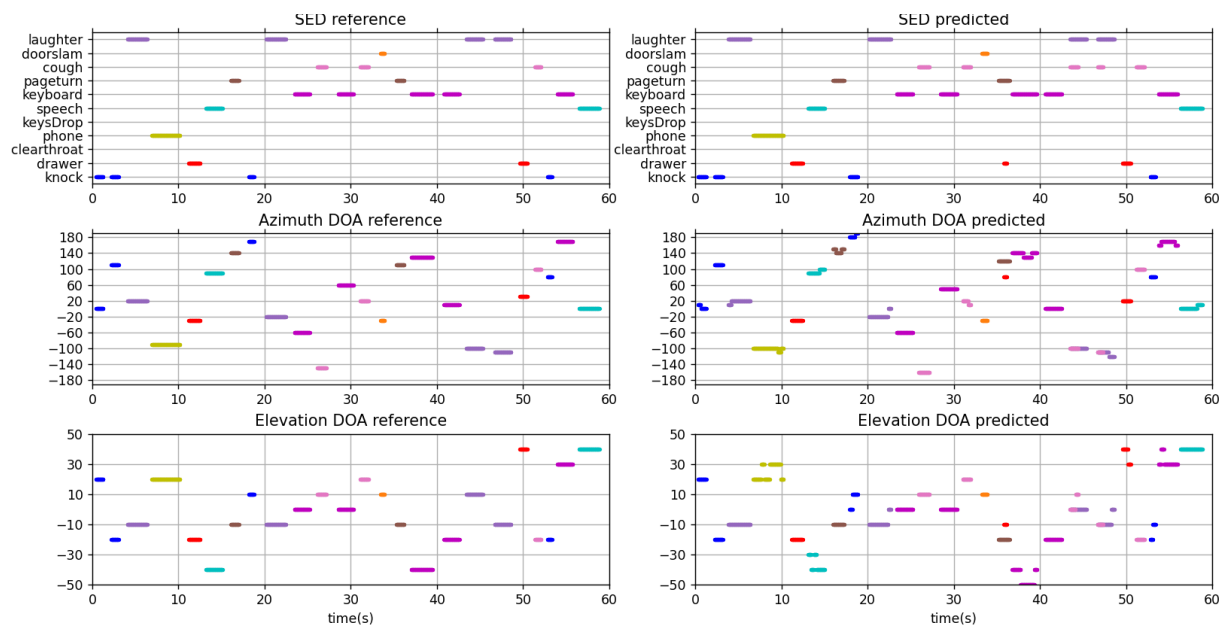
برای محیط پنجم:



شکل (۳-۱۱) نتایج بدست آمده برای split1_ir4_ov1_83.wav



شکل (۳-۱۲) نتایج بدست آمده برای split1_ir4_ov2_93.wav



شکل (۳-۱۳) نتایج بدست آمده برای split1_ir4_ov1_89.wav

فصل ٤

فصل ٤ :مراجع

مراجع

- [1] T. N. T. Nguyen, W.-S. Gan, R. Ranjan, and D. L. Jones, "Robust source counting and DoA estimation using spatial pseudo-spectrum and convolutional neural network," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, 28:2626–2637, 2020a.
- [2] Y. Cao, Q. Kong, T. Iqbal, F. An, W. Wang, and M. Plumbley, "Polyphonic sound event detection and localization using a two-stage strategy," In *Proc. Detection and Classification of Acoustic Scenes and Events Workshop (DCASE Workshop)*, New York, NY, October 2019b.
- [3] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [4] Kong, Q., Cao, Y., Iqbal, T., Wang, Y., Wang, W., and Plumbley, M. D, "PANNS: Large-scale pretrained audio neural networks for audio pattern recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 28:2880–2894, 2020b.
- [5] T. N. T. Nguyen, K. N. Watcharasupat, N. K. Nguyen, D. L. Jones, and W.-S. Gan, "SALSA: Spatial cue-augmented log spectrogram for polyphonic sound event localization and detection," *arXiv:2110.00275*, 2021.
- [6] C. Knapp and G. Carter, "The generalized correlation method for estimation of time delay," *IEEE Trans. Acoust., Speech, Signal Process.*, 24(4):320–327, Aug. 1976.
- [7] T. N. T. Nguyen, D. L. Jones, K. N. Watcharasupat, H. Phan, and W.-S. Gan, "Salsa-lite: A fast and effective feature for polyphonic sound event localization and detection with microphone arrays," in *IEEE Int. Conf. on Acoustics, Speech and Sig. Proc. (ICASSP)*, 2022, pp. 716–720.
- [8] Pierre-Amaury Grumiaux. Deep learning for speaker counting and localization with Ambisonics signals. Signal and Image processing. Université Grenoble Alpes [2020-..], 2021. English. ffNNT : 2021GRALT087ff. fftel-03538600v2ff
- [9] A. Mesaros, T. Heittola, and T. Virtanen. "Metrics for polyphonic sound event detection." *Applied Sciences* 6, no. 6 (2016): 162.
- [10] S. Adavanne, A. Politis, and T. Virtanen, "Direction of arrival estimation for multiple sound sources using convolutional recurrent neural network," in *26th European Signal Processing Conference (EUSIPCO)*, 2018.

-
- [11] R. Ranjan, S. Jayabalan, T. N. T. Nguyen, and W. S. Gan, “Sound event detection and direction of arrival estimation using residual net and recurrent neural networks,” in *Detection and Classification of Acoustic Scenes and Events 2019 Workshop (DCASE2019)*, New York, NY, USA, 2019.
- [12] S. Adavanne, A. Politis, and T. Virtanen, “A multi-room reverberant dataset for sound event localization and detection,” in *Proceedings of the Detection and Classification of Acoustic Scenes and Events 2019 Workshop (DCASE2019)*, New York University, NY, USA, October 2019, pp. 10–14

پیوست‌ها

برای استخراج ویژگی GCC-PHAT در کنار طیف log-mel میتوان از کد زیر استفاده کرد:

```
class LogMelGccExtractor():
    """
    Extract log-mel spectrograms and GCC-PHAT features.
    """
    def __init__(self, fs, nfft, hopsize, mel_bins, window, fmin):
        """
        fs: Sampling rate.
        nfft: Number of FFT points.
        hopsize: Number of samples for hopping.
        mel_bins: Number of mel bands.
        window: Type of window.
        fmin: Min frequency to extract feature (Hz).
        """
        self.fs = fs
        self.nfft = nfft
        self.hopsize = hopsize
        self.mel_bins = mel_bins
        self.window = window
        self.fmin = fmin
        self.melW = librosa.filters.mel(sr = fs, n_fft = nfft, n_mels = mel_bins, fmin = fmin)

    def logmel(self, sig):
        """
        Compute logmel of single channel signal
        sig: (n_samples,).
        return: S_logmel: (1, n_frames, n_mels).
        """
        S = np.abs(librosa.stft(y = sig,
                               n_fft = self.nfft,
                               hop_length = self.hopsize,
                               center = True,
                               window = self.window,
                               pad_mode = 'reflect'))
        S_mel = np.dot(self.melW, S ** 2).T
        S_logmel = librosa.power_to_db(S_mel, ref = 1.0, amin = 1e-10, top_db = None)
        S_logmel = np.expand_dims(S_logmel, axis = 0)

        return S_logmel
```

```
def gcc_phat(self, sig, refsig):
    """
    Compute GCC-PHAT between sig and refsig.
    sig: (n_samples,).
    refsig: (n_samples,).
    return: gcc_phat: (1, n_frames, n_mels)
    """
    ncorr = 2 * self.nfft - 1
    nfft = int(2 ** np.ceil(np.log2(np.abs(ncorr)))) # this nfft double the length of win_length
    Px = librosa.stft(y = sig,
                      n_fft = nfft,
                      hop_length = self.hopsize,
                      center = True,
                      window = self.window,
                      pad_mode = 'reflect')

    Px_ref = librosa.stft(y = refsig,
                          n_fft = nfft,
                          hop_length = self.hopsize,
                          center = True,
                          window = self.window,
                          pad_mode = 'reflect')

    R = Px * np.conj(Px_ref)

    n_frames = R.shape[1]
    gcc_phat = []
    for i in range(n_frames):
        spec = R[:, i].flatten()
        cc = np.fft.irfft(np.exp(1.j*np.angle(spec)))
        cc = np.concatenate((cc[-self.mel_bins // 2 :], cc[: self.mel_bins // 2]))
        gcc_phat.append(cc)
    gcc_phat = np.array(gcc_phat)
    gcc_phat = gcc_phat[None, :, :]

    return gcc_phat
```

```
def extract(self, audio):
    """
    audio: (n_channels, n_samples).
    return: features (n_channel + n_channels*(n_channel-1)/2, n_timeframes, n_features).
    """
    n_channels = audio.shape[0]
    logmel_features = []
    gccPhat_features = []
    for n in range(n_channels):
        logmel_features.append(self.logmel(audio[n]))
        for m in range(n + 1, n_channels):
            gccPhat_features.append(
                self.gcc_phat(sig = audio[m], refsig = audio[n]))
    logmel_features = np.concatenate(logmel_features, axis=0)
    gccPhat_features = np.concatenate(gccPhat_features, axis=0)
    feature = np.concatenate([logmel_features, gccPhat_features])
    return feature
```

و برای استخراج ویژگی جدید معرفی شده، با تعاریف زیر:

```
n_bins = nfft // 2 + 1
lower_bin0 = np.int32(np.floor(fmin * nfft / np.float32(fs))) # 512: 1; 256: 0
lower_bin = np.max((1, lower_bin0))
# Cutoff frequency for spectrograms
fmax = 8000 # Hz
cutoff_bin = np.int32(np.floor(fmax * nfft / np.float32(fs))) # 8000 Hz, 512 nfft: cutoff_bin = 192
# Normalization factor for salsa_lite
c = 343
delta = 2 * np.pi * fs / (nfft * c)
```

کد پیاده سازی آن بصورت زیر خواهد شد:

```
class SalsaliteExtractor():
    """
    Extract salsa_lite features(only for MIC format!):
        salsa_lite: log-linear spectrogram + normalized interchannel phase difference
    The frequency range of log-linear spectrogram is 0 to 8KHz.
    """
    def __init__(self, fs, nfft, hopsize, window, fmin, fmax, nmics):
        """
        fs: Sampling rate.
        nfft: Number of FFT points.
        hopsize: Number of samples for hopping.
        mel_bins: Number of mel bands.
        window: Type of window.
        fmin: Min frequency to extract feature (Hz).
        fmax: Max frequency to extract feature (Hz).
        nmics: Number of mics (channels).
        """
        self.fs = fs
        self.nfft = nfft
        self.hopsize = hopsize
        self.window = window
        self.fmin = fmin
        self.fmax = fmax
        self.nmics = nmics
```

```
def extract(self, audio) :
    """
    audio: (n_channels, n_samples).
    return: features (n_channel + n_channels-1), n_timeframes, n_features).
    """
    # for normalization factor
    freq_vector = np.arange(n_bins)
    freq_vector[0] = 1
    freq_vector = freq_vector[:, None, None] # n_bins x 1 x 1

    # Compute stft
    log_specs = []
    for imic in np.arange(self.nmics):
        stft = librosa.stft(y = audio[imic, :], n_fft=self.nfft, hop_length=self.hopsize,
                            center=True, window=self.window, pad_mode='reflect')
        if imic == 0:
            n_frames = stft.shape[1]
            X = np.zeros((n_bins, n_frames, self.nmics), dtype='complex') # (n_bins, n_frames, n_mics)
            X[:, :, imic] = stft
        # Compute log linear power spectrum
        spec = (np.abs(stft) ** 2).T
        log_spec = librosa.power_to_db(spec, ref = 1.0, amin = 1e-10, top_db = None)
        log_spec = np.expand_dims(log_spec, axis = 0)
        log_specs.append(log_spec)
    log_specs = np.concatenate(log_specs, axis = 0) # (n_mics, n_frames, n_bins)

    # Compute spatial feature
    phase_vector = np.angle(X[:, :, 1:] * np.conj(X[:, :, 0, None]))
    phase_vector = phase_vector / (delta * freq_vector)
    phase_vector = np.transpose(phase_vector, (2, 1, 0)) # (n_mics, n_frames, n_bins)
    # Crop frequency
    log_specs = log_specs[:, :, lower_bin:cutoff_bin]
    phase_vector = phase_vector[:, :, lower_bin:cutoff_bin]

    # Stack features
    audio_feature = np.concatenate((log_specs, phase_vector), axis=0)

    return audio_feature
```

باتوجه به اینکه در سیستم در یک شاخه فقط خطای SED و در یک شاخه فقط خطای DOAE برای ما اهمیت دارد، تابع loss را بصورت زیر تعریف کنیم:

```
def hybrid_regr_loss(output_dict, target_dict, task_type, loss_type='MSE'):
    """
    Hybrid loss for regression doa:
    Input:
        output_dict: predict dictionary
        target_dict: target dictionary
        task_type: 'sed_only' | 'doa_only' | 'two_staged_eval' | 'seld'
        loss_type: 'MSE' | 'MAE'
    """
    class_num = target_dict['events'].shape[-1]
    sed_loss = binary_cross_entropy(
        output=output_dict['events'],
        target=target_dict['events']
    )
    azimuth_loss = mean_error(
        output=output_dict['doas'],
        target=target_dict['doas'],
        mask=target_dict['events'],
        loss_type=loss_type
    )
    elevation_loss = mean_error(
        output=output_dict['doas'],
        target=target_dict['doas'],
        mask=target_dict['events'],
        loss_type=loss_type
    )
    if task_type == 'sed_only':
        beta = 0
    elif task_type == 'two_staged_eval' or task_type == 'doa_only':
        beta = 1
    elif task_type == 'seld':
        beta = 0.2
    doa_loss = elevation_loss + azimuth_loss
    seld_loss = (1 - beta) * sed_loss + beta * doa_loss
    return seld_loss, sed_loss, doa_loss
```

که تابع mean_error استفاده شده در آن بصورت زیر تعریف می شود:

```
def mean_error(output, target, mask, loss_type='MSE'):
    """
    # Align the time_steps of output and target
    N = min(output.shape[1], target.shape[1])
    output = output[:, 0: N, :]
    target = target[:, 0: N, :]
    mask = mask[:, 0: N, :]
    normalize_value = torch.sum(mask)
    if loss_type == 'MAE':
        out = torch.sum(torch.abs(output - target) * mask) / normalize_value
    elif loss_type == 'MSE':
        out = torch.sqrt(torch.sum((output - target)**2 * mask) / normalize_value)
    return out
```

باتوجه به این تابع میتوانیم هر دو مسئله را با هم نیز حل کنیم اما همانطور که گفته شد به نتایج بدتری می رسیم. ساختار سیستم هم بصورت زیر تعریف می شود (برای شاخه DOAE از کلاس pretrained استفاده می شود)

```

class MobileNetV1(nn.Module):
    def __init__(self, class_num, pool_type='avg', pool_size=(2,2), interp_ratio=16, pretrained_path=None):
        super().__init__()

        self.class_num = class_num
        self.pool_type = pool_type
        self.pool_size = pool_size
        self.interp_ratio = interp_ratio

    def conv_bn(inp, oup, stride):
        _layers = [
            nn.Conv2d(inp, oup, 3, 1, 1, bias=False),
            nn.AvgPool2d(stride),
            nn.BatchNorm2d(oup),
            nn.ReLU(inplace=True)
        ]
        _layers = nn.Sequential(*_layers)
        init_layer(_layers[0])
        init_bn(_layers[2])
        return _layers

    def conv_dw(inp, oup, stride):
        _layers = [
            nn.Conv2d(inp, inp, 3, 1, 1, groups=inp, bias=False),
            nn.AvgPool2d(stride),
            nn.BatchNorm2d(inp),
            nn.ReLU(inplace=True),
            nn.Conv2d(inp, oup, 1, 1, 0, bias=False),
            nn.BatchNorm2d(oup),
            nn.ReLU(inplace=True)
        ]
        _layers = nn.Sequential(*_layers)
        init_layer(_layers[0])
        init_bn(_layers[2])
        init_layer(_layers[4])
        init_bn(_layers[5])
        return _layers

```

```

def __init__(self, class_num, pool_type='avg', pool_size=(2,2), interp_ratio=16, pretrained_path=None):
    self.features = nn.Sequential(
        conv_bn( 7, 32, 2),
        conv_dw( 32, 64, 1),
        conv_dw( 64, 128, 2),
        conv_dw(128, 128, 1),
        conv_dw(128, 256, 2),
        conv_dw(256, 256, 1),
        conv_dw(256, 256, 1),
        conv_dw(256, 256, 1),
        conv_dw(256, 256, 1),
        conv_dw(256, 512, 2),
        conv_dw(512, 512, 1))

    self.gru = nn.GRU(input_size=512, hidden_size=256,
        num_layers=2, dropout=0.3, batch_first=True, bidirectional=True)
    #self.gru = nn.GRU(input_size=512, hidden_size=256,
    # num_layers=1, batch_first=True, bidirectional=True)
    #self.event_fc = nn.Linear(512, class_num, bias=True)
    #self.azimuth_fc = nn.Linear(512, class_num, bias=True)
    #self.elevation_fc = nn.Linear(512, class_num, bias=True)
    self.event_fc1 = nn.Linear(512, 256, bias=True)
    self.event_dp_1 = nn.Dropout(p=0.2)
    self.azimuth_fc1 = nn.Linear(512, 256, bias=True)
    self.azimuth_dp_1 = nn.Dropout(p=0.2)
    self.elevation_fc1 = nn.Linear(512, 256, bias=True)
    self.elevation_dp_1 = nn.Dropout(p=0.2)

    self.event_fc2 = nn.Linear(256, class_num, bias=True)
    self.event_dp_2 = nn.Dropout(p=0.2)
    self.azimuth_fc2 = nn.Linear(256, class_num, bias=True)
    self.azimuth_dp_2 = nn.Dropout(p=0.2)
    self.elevation_fc2 = nn.Linear(256, class_num, bias=True)
    self.elevation_dp_2 = nn.Dropout(p=0.2)

    self.init_weights()

def init_weights(self):
    init_gru(self.gru)
    init_layer(self.event_fc)
    init_layer(self.azimuth_fc)
    init_layer(self.elevation_fc)
    init_layer(self.event_fc1)
    init_layer(self.azimuth_fc1)
    init_layer(self.elevation_fc1)
    init_layer(self.event_fc2)
    init_layer(self.azimuth_fc2)
    init_layer(self.elevation_fc2)

```

```

def forward(self, x):
    '''input: (batch_size, mic_channels, time_steps, mel_bins)'''

    x = self.features(x)
    '''(batch_size, feature_maps, time_steps, mel_bins)'''

    if self.pool_type == 'avg':
        x = torch.mean(x, dim=3)
    elif self.pool_type == 'max':
        (x, _) = torch.max(x, dim=3)
    '''(batch_size, feature_maps, time_steps)'''

    x = x.permute(0, 2, 1)
    '''(batch_size, time_steps, feature_maps)'''

    (x, _) = self.gru(x)

    #event_output = torch.sigmoid(self.event_fc(x))
    #azimuth_output = self.azimuth_fc(x)
    #elevation_output = self.elevation_fc(x)
    '''(batch_size, time_steps, class_num)'''
    event_output = F.relu(self.event_fc1(self.event_dp_1(x)))
    event_output = torch.sigmoid(self.event_fc2(self.event_dp_2(event_output)))
    azimuth_output = F.relu(self.azimuth_fc1(self.azimuth_dp_1(x)))
    azimuth_output = self.azimuth_fc2(self.azimuth_dp_2(azimuth_output))
    elevation_output = F.relu(self.elevation_fc1(self.elevation_dp_1(x)))
    elevation_output = self.elevation_fc2(self.elevation_dp_2(elevation_output))

    # Interpolate
    event_output = interpolate(event_output, self.interp_ratio)
    azimuth_output = interpolate(azimuth_output, self.interp_ratio)
    elevation_output = interpolate(elevation_output, self.interp_ratio)

    output = {
        'events': event_output,
        'doas': torch.cat((azimuth_output, elevation_output), dim=-1)
    }

    return output

```

```

class pretrained_MobileNetV1(MobileNetV1):
    def __init__(self, class_num, pool_type='avg', pool_size=(2,2), interp_ratio=16, pretrained_path=None):
        super().__init__(class_num, pool_type, pool_size, interp_ratio, pretrained_path)
        self.load_weights(pretrained_path)

    def load_weights(self, pretrained_path):
        model = MobileNetV1(self.class_num, self.pool_type, self.pool_size, self.interp_ratio)
        checkpoint = torch.load(pretrained_path, map_location=torch.device('cpu'), storage_loc=storage)
        model.load_state_dict(checkpoint['model_state_dict'])

        self.features = model.features
        #
        self.gru = nn.GRU(input_size=512, hidden_size=256,
                           num_layers=1, batch_first=True, bidirectional=True)

        init_gru(self.gru)
        #init_layer(self.event_fc)
        #init_layer(self.azimuth_fc)
        #init_layer(self.elevation_fc)
        init_layer(self.event_fc1)
        init_layer(self.azimuth_fc1)
        init_layer(self.elevation_fc1)
        init_layer(self.event_fc2)
        init_layer(self.azimuth_fc2)
        init_layer(self.elevation_fc2)

```



University of Tehran



College of Engineering

School of Electrical and Computer Engineering

DOA Estimation Using Neural Network

By:

AhmadReza Shahsavari

Supervisor:

Dr. Ali Olfat